

1. Angular Framework

- Q1. What is Angular?
- Q2. What are Angular advantages? [View Answer](#)
- Q3. What is the difference between AngularJS and Angular?
- Q4. What is NPM?
- Q5. What is CLI tool?

2. Components & Modules

- Q6. What are Components in Angular?
- Q7. What is a Selector and Template?
- Q8. What is Module in Angular? What is app.module.ts file?
- Q9. How an Angular App gets Loaded and Started? What are index.html, app-root, selector and main.ts? [View Answer](#)
- Q10. What is a Bootstrapped Module & Bootstrapped Component?

3. Data Binding

- Q11. What is Data Binding in Angular?
- Q12. What is String Interpolation in Angular?
- Q13. What is Property Binding in Angular?
- Q14. What is Event Binding in Angular?
- Q15. What is Two way Binding in Angular? [View Answer](#)

4. Directives

- Q16. What are Directives? What are the type of directives?
- Q17. What is *ngIf Structural directive?
- Q18. What is *ngFor Structural directive?
- Q19. What is *ngSwitch Structural directive?
- Q20. What is [ngStyle] Attribute directive?
- Q21. What is [ngClass] Attribute directive?
- Q22. What is the difference between Component, Attribute and Structural Directives? [View Answer](#)

5. Decorator & Pipes

- Q23. What is Decorator?
- Q24. What are the types of Decorator?
- Q25. What are Pipes? What are the types of Pipes & Parameterized Pipes? [View Answer](#)
- Q26. What is Chaining Pipes?

6. Services & Dependency Injection

- Q27. Explain Services with Example? [View Answer](#)
- Q28. How to create Service in Angular?
- Q29. How to use Dependency Injector with Services in Angular?
- Q30. What is Hierarchical Dependency Injection? [View Answer](#)
- Q31. What is Provider in Angular?
- Q32. What is the role of @Injectable Decorator in a Service?

7. Lifecycle-Hooks

- Q33. What are Parent-Child Components?
- Q34. What are Lifecycle Hooks in Angular?
- Q35. What is a Constructor in Angular?
- Q36. What is ngOnInit life cycle hook in Angular?
- Q37. What is the difference between constructor and ngOnInit? [View Answer](#)

8. Observable\ HttpClient\ RxJS

- Q38. What are Asynchronous operations?
- Q39. What is the difference between Promise and Observable? [View Answer](#)
- Q40. What is RxJS?
- Q41. What is Observable? How to implement Observable [View Answer](#)
- Q42. What is the role of HttpClient in Angular?

9. Typescript-Basics

- Q43. What is Typescript? Or What is the difference between Typescript and Javascript?
- Q44. What is the difference between let and var keyword?
- Q45. What is Type annotation?
- Q46. What are Built in/ Primitive and User-Defined/ Non-primitive Types in Typescript?
- Q47. What is "any" type in Typescript? [View Answer](#)
- Q48. What is Enum type in Typescript?
- Q49. What is Type Assertion in Typescript?
- Q50. What are Arrow Functions in Typescript?

1. Angular Framework ✓

Q1. What is Angular?

Q2. What are Angular advantages? **V. IMP.**

Q3. What is the difference between AngularJS and Angular?

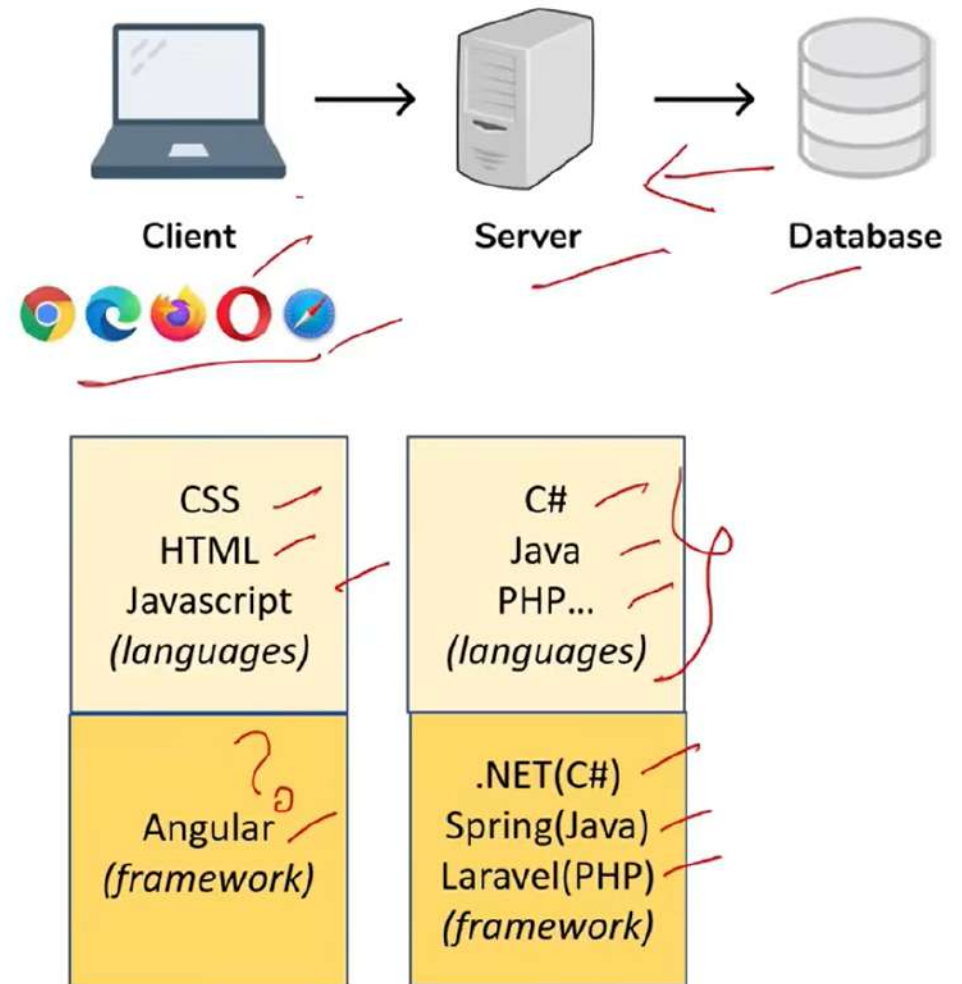
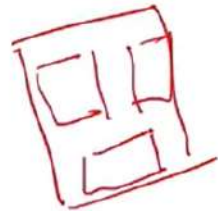
Q4. What is NPM?

Q5. What is CLI tool?



What is Angular?

- ❖ Angular is a component-based framework for building structured, scalable and single page web applications for client side.



What are Angular advantages?



It is Relatively simple to build Single Page Applications (Components)



To make flexible and structured applications (OOPS friendly)



It is Cross platform and Open Source (Free)

O.S .



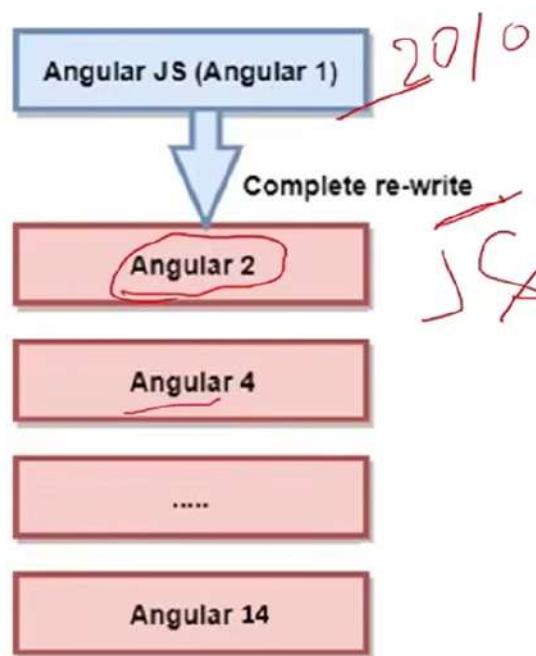
Reusable Code (Services)



Testability (spec.ts)

U . !

What is the difference between Angular and Angular JS? OR What are the advantages of Angular over AngularJS?



Angular JS	Angular
1. It only supports JavaScript.	It support both JavaScript and TypeScript.
2. This framework has a model-view-controller (MVC) architecture.	This framework has a component based architecture. <i>fast</i>
3. It does not have CLI tool.	It has CLI tool.
4. It does not use Dependency Injection.	It uses Dependency Injection. <i>DI</i>
5. It does not support mobile browsers.	It also support mobile browsers. ✓
6. It is not so fast	It is very fast

FileEditSelectionViewGoRunTerminalHelp

InterviewHappy - Visual Studio Code

EXPLORER

INTERVIEWHAPPY

> .vscode

> node_modules

> src

> app

TS app-routing.module.tsA

app.component.cssA

<> app.component.htmlA

TS app.component.spec.tsA

TS app.component.tsA

TS app.module.tsA

> assets

> environments

★ favicon.icoA

<> index.htmlA

TS main.tsA

TS polyfills.tsA

styles.cssA

TS test.tsA

≡ .browserslistrcA

> OUTLINE

> TIMELINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

powershell + - [] [X] ^ X

PS C:\InterviewHappy> ng generate component TestComponetI

master+ 0 0 0

SUBSCRIBE

What is a Selector and Template?

A selector is used to identify each component uniquely into the component tree.

```
Terminal  Help  app.component.ts - first-project - Visual Studio Code

TS app.component.ts A X
src > app > TS app.component.ts > ...
1  import { Component } from '@angular/core';
2
3  @Component({
4    selector: 'app-root',
5    templateUrl: './app.component.html',
6    styleUrls: ['./app.component.css']
7  })
8  export class AppComponent {
9    title = 'first-project';
10 }
11
```

```
Terminal  Help  index.html - first-project - Visual Studio Code

index.html M X
c > <> index.html > ...
1  <!doctype html>
2  <html lang="en">
3  <head>
4    <meta charset="utf-8">
5    <title>FirstProject</title>
6    <base href="/">
7    <meta name="viewport" content="width=device-width, initial-scale=1">
8    <link rel="icon" type="image/x-icon" href="favicon.ico">
9  </head>
10 <body>
11
12   <app-root></app-root>
13
14 </body>
15 </html>
```



What is Module in Angular? What is app.module.ts file?



- ❖ Module is a place where you can group the components, directives, pipes, and services, which are related to the application.



EXPLORER

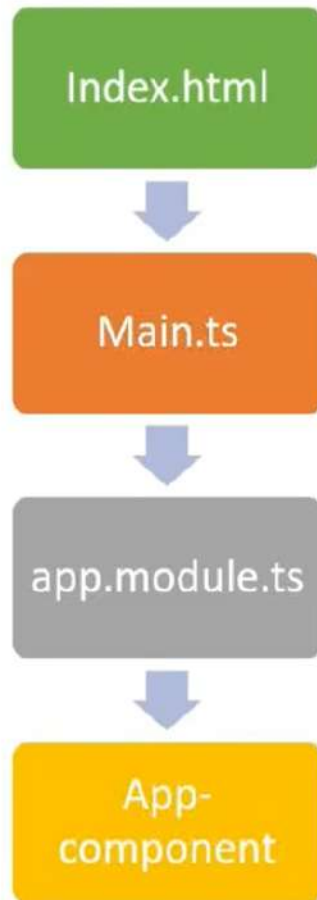
FIRST-PROJECT

- tasks.json A
- node_modules
- src
 - app
 - app-routing.module.ts A
 - app.component.css M
 - app.component.html A
 - app.component.spec.ts A
 - app.component.ts A
 - app.module.ts A
 - assets
 - .gitkeep A
 - environments
 - environment.prod.ts A
 - environment.ts A
 - favicon.ico A
 - index.html A
 - main.ts A

OUTLINE

TIMELINE

```
TS app.module.ts A X
src > app > TS app.module.ts > ...
1  import { NgModule } from '@angular/core';
2  import { BrowserModule } from '@angular/platform-browser';
3
4  import { AppRoutingModule } from './app-routing.module';
5  import { AppComponent } from './app.component';
6
7  @NgModule({
8    declarations: [
9      AppComponent
10   ],
11   imports: [
12     BrowserModule,
13     AppRoutingModule
14   ],
15   providers: [],
16   bootstrap: [AppComponent]
17 })
18 export class AppModule { }
19
```



- ❖ Angular is used to create Single Page Applications. index.html file is that single page. Index.html will invoke main.js file which is the javascript version of main.ts file.
- ❖ main.ts file is like the entry point of web-app. It compiles the web-app and bootstraps the AppModule to run in the browser.
- ❖ App module file will then bootstrap the Appcomponent.
- ❖ App-component or app-root component is the html which you will see finally.



What is a Bootstrapped Module & Bootstrapped Component?



- ❖ When the Angular web application will start then the first **module launched** is the bootstrapped module and same is true for the bootstrapped component also.



3. Data Binding ✓

Q11. What is Data Binding in Angular?

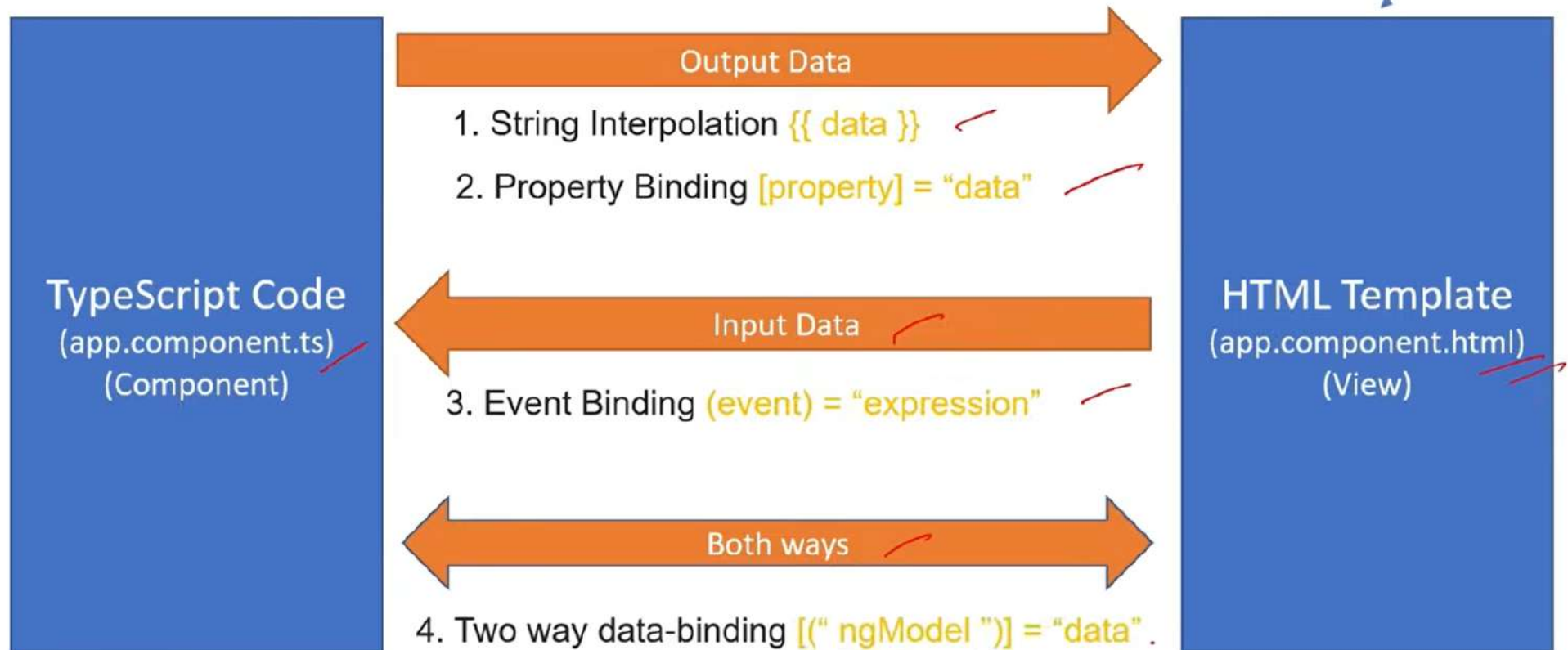
Q12. What is String Interpolation in Angular?

Q13. What is Property Binding in Angular?

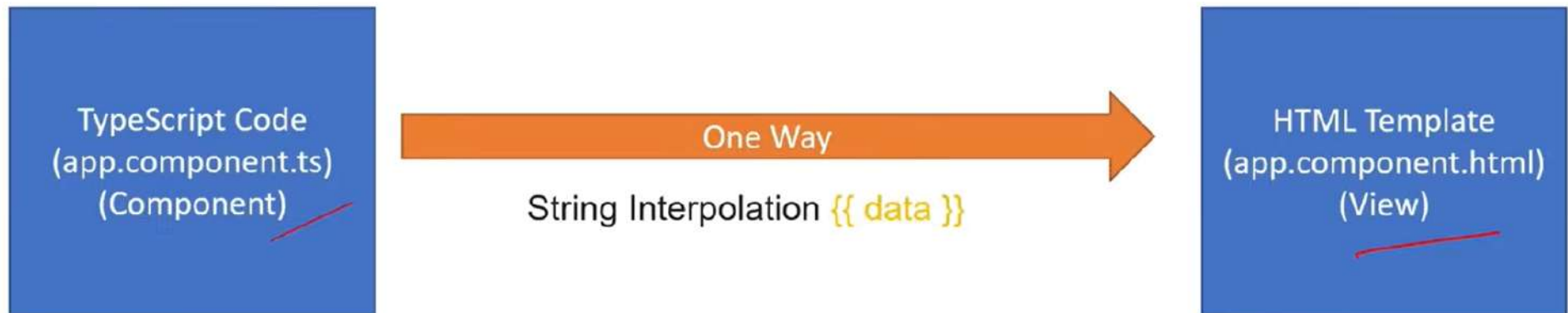
Q14. What is Event Binding in Angular?

Q15. What is Two way Binding in Angular? **V. IMP.**

- ❖ Data binding is the way to communicate between your typescript code of your component and your html view.



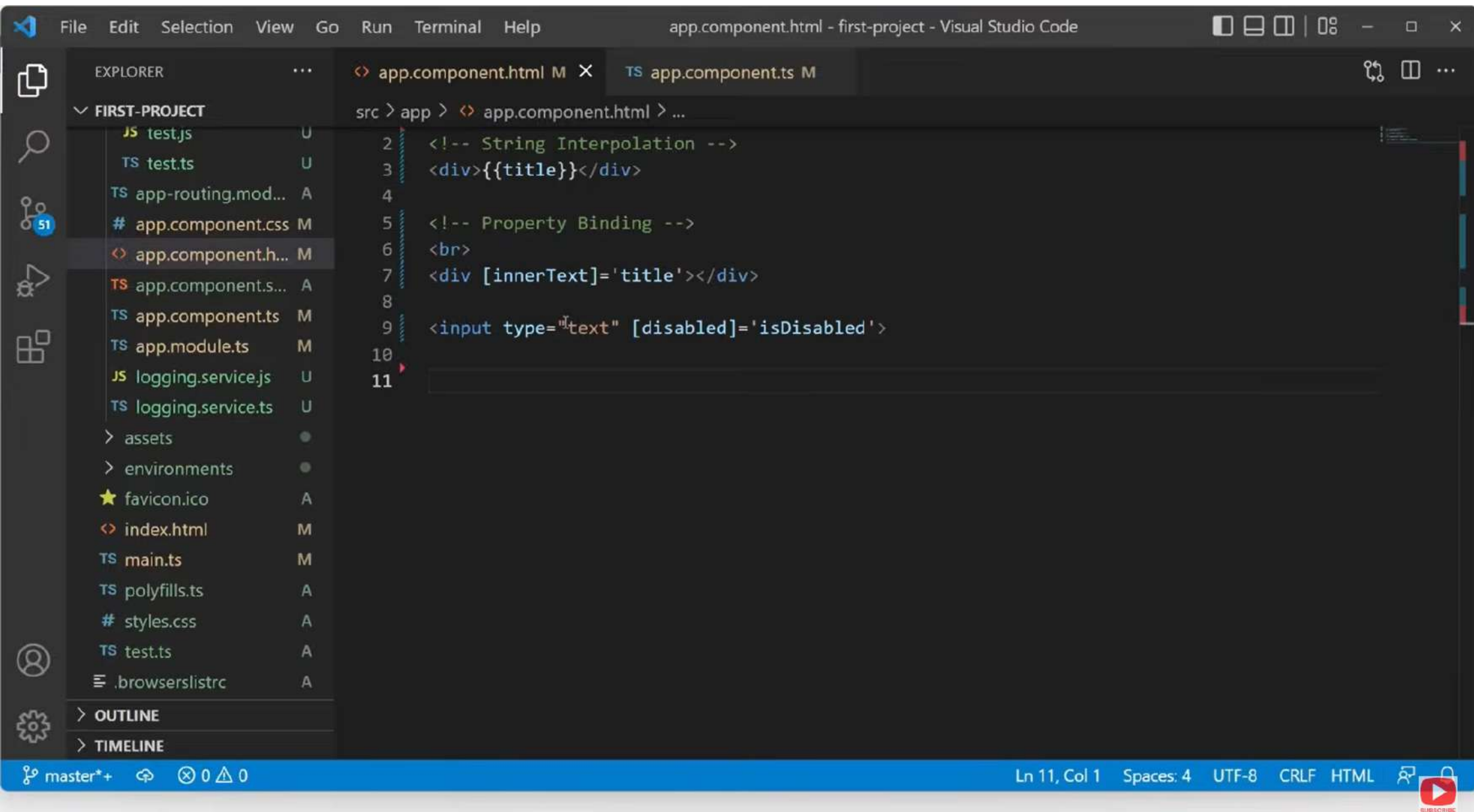
- ❖ String Interpolation is a one-way data-binding technique that is used to transfer the data from a TypeScript code(component) to an HTML template (view).



1. String interpolation can work on string type only not numbers or any other type.

- ❖ Property binding is a superset of interpolation. It can do whatever interpolation can do. In addition, it can set an element property to a non-string data value like Boolean.





- ❖ Event binding is used to handle the events raised by the user actions like button click.



- ❖ Two-way data binding in Angular will help users to exchange data from the view to component and then from component to the view at the same time.

← → ↻ ⓘ localhost:4200

Input:

We Typed:

Hello wor

TypeScript Code
(app.component.ts)
(Component)



HTML Template
(app.component.html)
(View)

Two way data-binding [(" ngModel ")] = "data"

4. Directives

Q16. What are Directives? What are the type of directives?

Q17. What is *ngIf Structural directive?

Q18. What is *ngFor Structural directive?

Q19. What is *ngSwitch Structural directive?

Q20. What is [ngStyle] Attribute directive?

Q21. What is [ngClass] Attribute directive?

Q22. What is the difference between Component, Attribute and Structural Directives? **V. IMP.**

- ❖ Directives are classes that add additional behavior to elements in your Angular applications.

Submit

Types of Directives

Structural

*ngIf

*ngFor

*ngSwitch

Attribute

[ngClass]

[ngStyle]

Component

Custom

Sunday

Monday

Tuesday

Wednesday

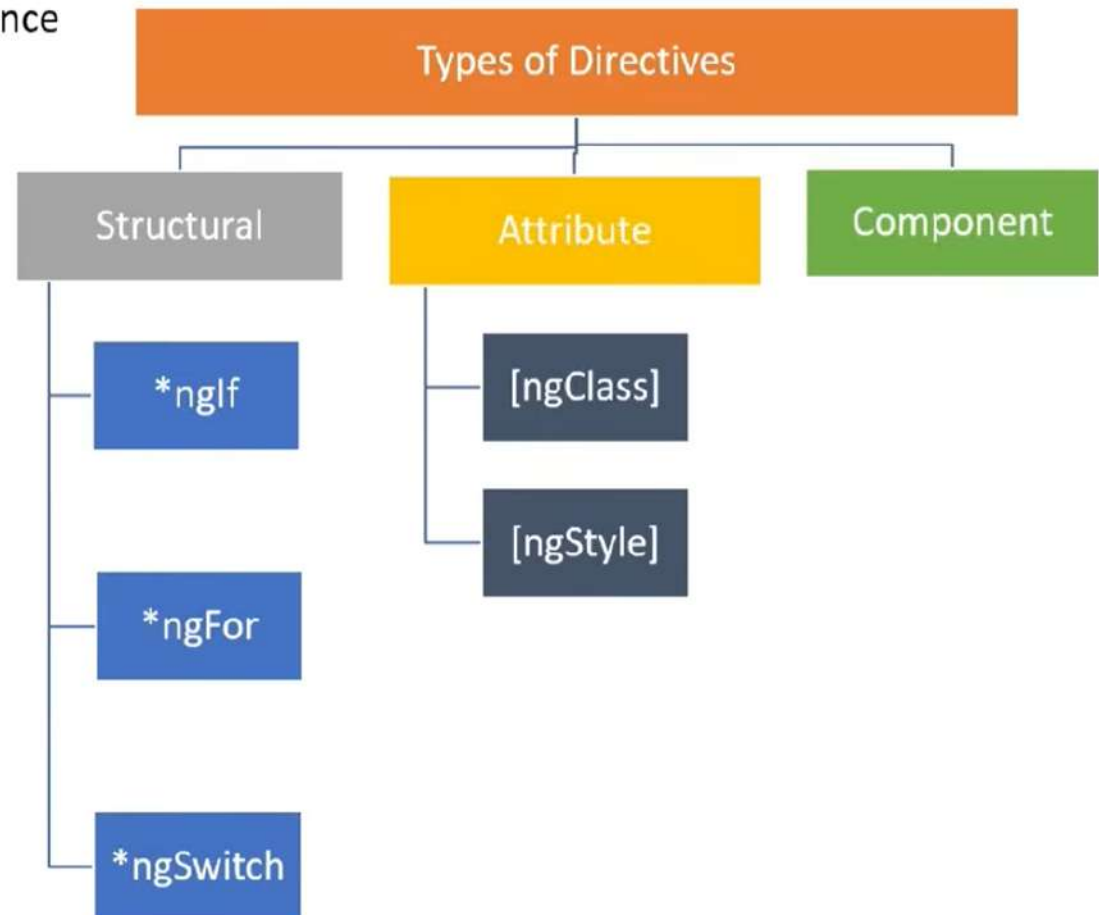
Structural directives change appearance of DOM by adding or removing elements.

Attribute directives change the appearance or behavior of an element.

Component directive are directives with its own template.

- ❖ Structural Directives are classes that change appearance of DOM by adding or removing elements.

- ❖ ngIf Directive is used to add or remove items based on the if condition.



EXPLORER

▼ FIRST-PROJECT

- JS test.js U
- TS test.ts U
- TS app-routing.module.ts A
- # app.component.css M
- <> app.component.html M
- TS app.component.spec.ts A
- TS app.component.ts M
- TS app.module.ts M
- JS logging.service.js U
- TS logging.service.ts U
- > assets
- > environments
- ★ favicon.ico A
- <> index.html M
- TS main.ts M
- TS polyfills.ts A
- # styles.css A
- TS test.ts A
- ≡ .browserslistrc A

> OUTLINE

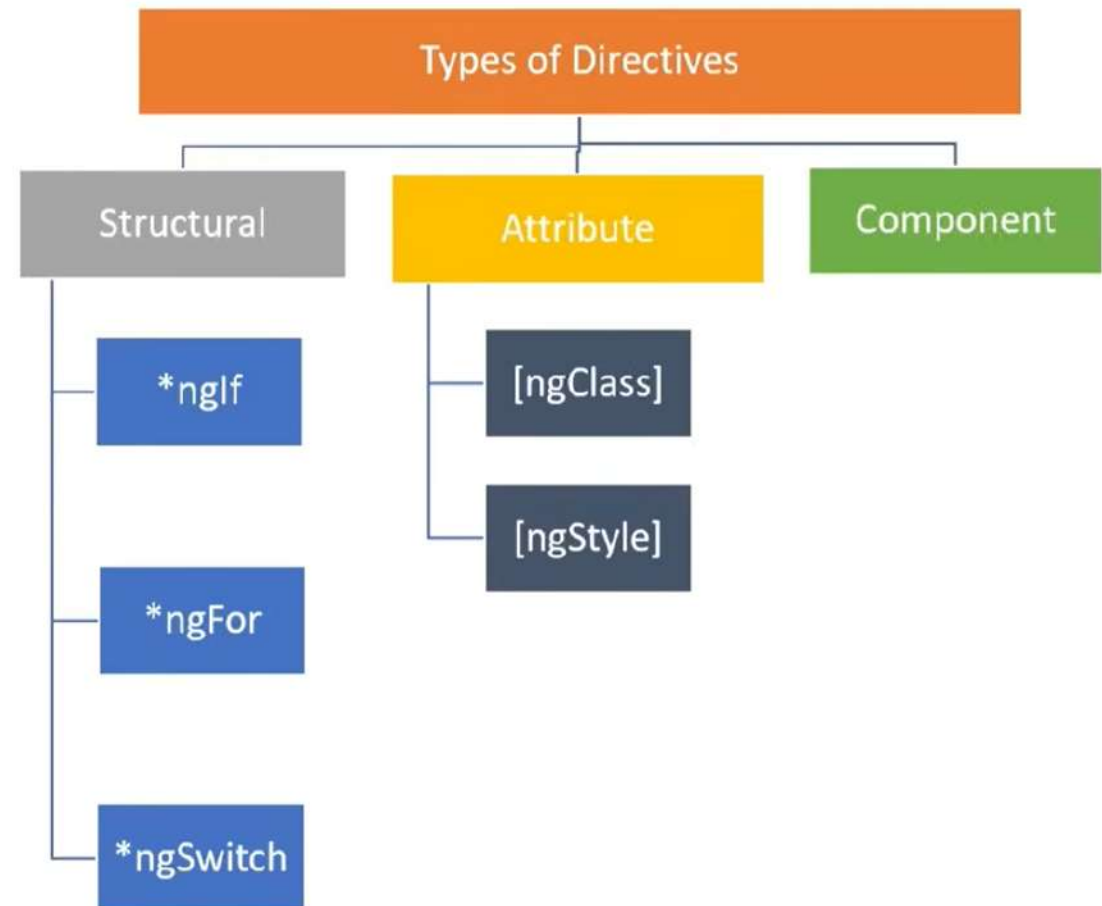
> TIMELINE

app.component.html M X TS app.component.ts M

src > app > <> app.component.html > ul > li

Go to component

```
1 <ul >
2   <li *ngIf="students.length > 0">
3     Two Students
4   </li>
5   <li *ngIf="students.length == 0">
6     Zero Students
7   </li>
8 </ul>
9
10
```



- ❖ The *ngFor directive is used to iterate a list of items and then display them.

EXPLORER

▼ FIRST-PROJECT

- JS test.js U
- TS test.ts U
- TS app-routing.module.ts A
- # app.component.css M
- <> app.component.html M
- TS app.component.spec.ts A
- TS app.component.ts M
- TS app.module.ts M
- JS logging.service.js U
- TS logging.service.ts U
- > assets ●
- > environments ●
- ★ favicon.ico A
- <> index.html M
- TS main.ts M
- TS polyfills.ts A
- # styles.css A
- TS test.ts A
- ≡ .browserslistrc A

> OUTLINE

> TIMELINE

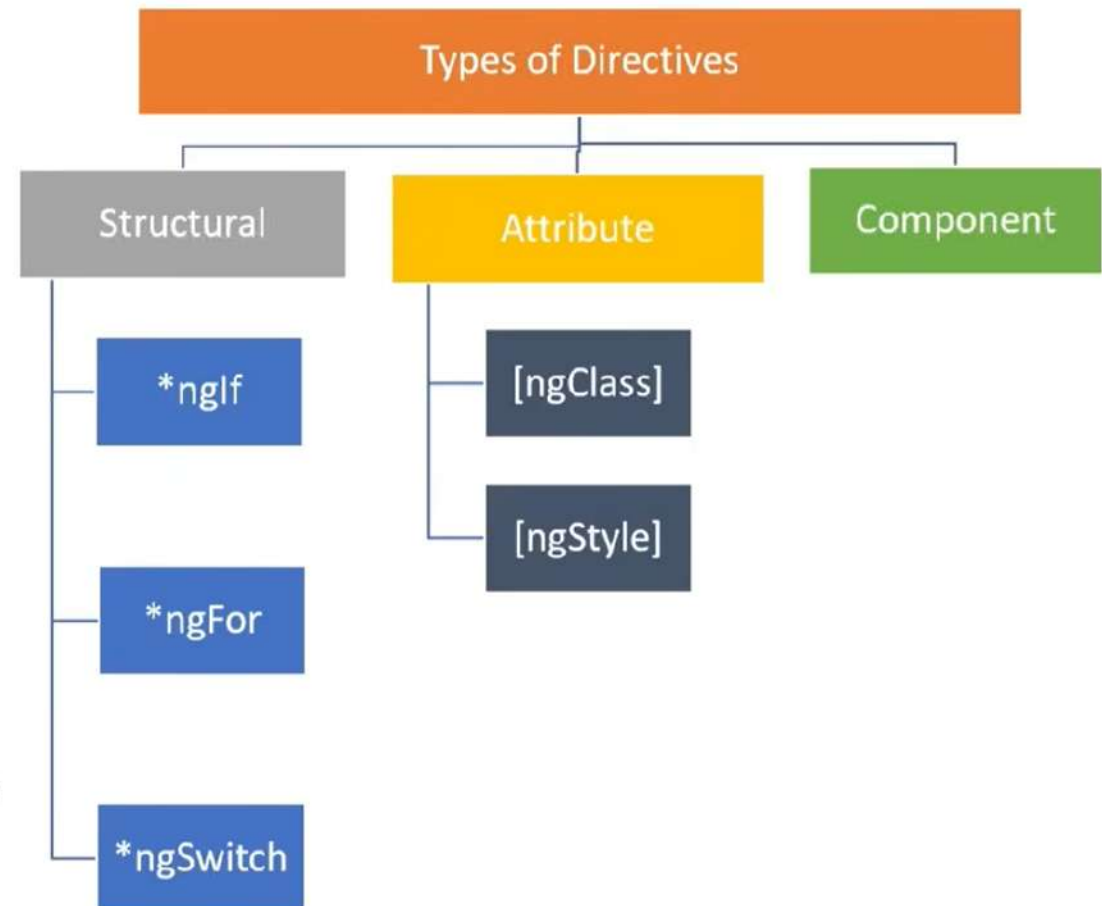
<> app.component.html M X TS app.component.ts M

src > app > <> app.component.html > li

Go to component

```
1
2 <li *ngFor="let student of studentList;">
3   | {{student}}
4 </li>
5
6
```





- ❖ ngSwitch is an structural directive used in combination with *ngSwitchCase and *ngSwitchDefault that are both structural directives.

EXPLORER

▼ FIRST-PROJECT

- JS test.js U
- TS test.ts U
- TS app-routing.module.ts A
- # app.component.css M
- <> app.component.html M
- TS app.component.spec.ts A
- TS app.component.ts M
- TS app.module.ts M
- JS logging.service.js U
- TS logging.service.ts U
- > assets
- > environments
- ★ favicon.ico A
- <> index.html M
- TS main.ts M
- TS polyfills.ts A
- # styles.css A
- TS test.ts A
- ≡ .browserslistrc A

> OUTLINE

> TIMELINE

<> app.component.html M X TS app.component.ts M

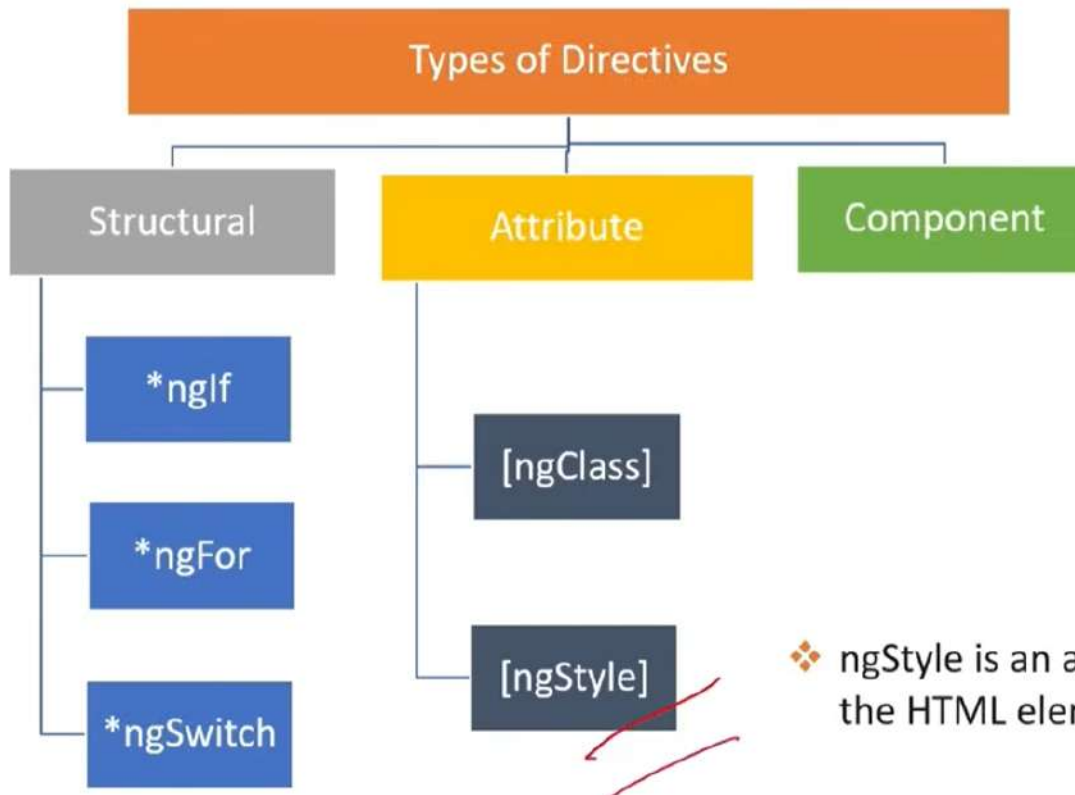
src > app > <> app.component.html > ul

Go to component

```
1 <ul [ngSwitch]="studentList.length">
2   <li *ngSwitchCase=0>Zero Student</li>
3   <li *ngSwitchCase=1>One Student</li>
4   <li *ngSwitchCase=2>Two Students</li>
5   <li *ngSwitchDefault>More Students</li>
6 </ul>
```

I

- ❖ Attribute directives change the appearance or behavior of an element.



- ❖ ngStyle is an attribute directive that updates styles for the HTML element.

EXPLORER

FIRST-PROJECT

JS test.js U

TS test.ts U

TS app-routing.mod... A

app.component.css M

<> app.component.h... M

TS app.component.s... A

TS app.component.ts M

TS app.module.ts M

JS logging.service.js U

TS logging.service.ts U

> assets ●

> environments ●

★ favicon.ico A

<> index.html M

TS main.ts M

TS polyfills.ts A

styles.css A

TS test.ts A

≡ .browserslistrc A

> OUTLINE

> TIMELINE

src > app > <> app.component.html > div

Go to component

1 <!-- ngStyle is an attribute directive that updates styles for the HTML element. -->

2

3 <div style ="background-color: red;">I am Happy</div>

4

5

6 <div [ngStyle]="{'background-color':'red'}">I am ngStyle Happy</div>

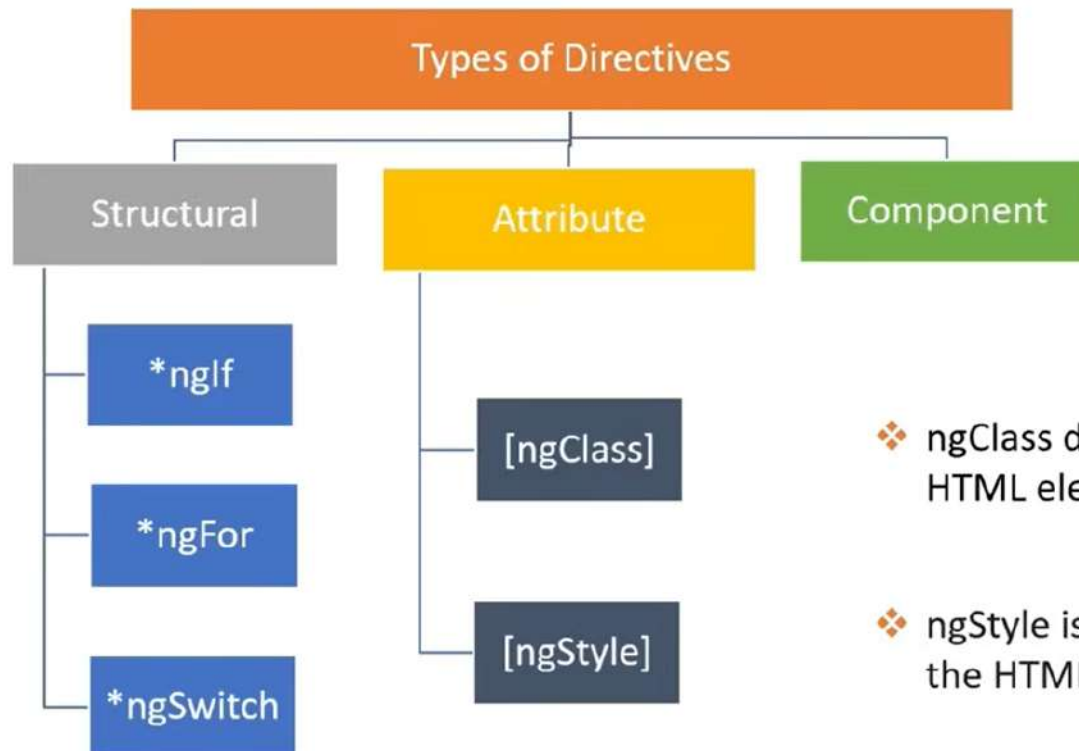
7

8

9 <div [ngStyle]="{'background-color': colorName}">ngStyle background</div>

master*+ 0 0 Ln 9, Col 14 (7 selected) Spaces: 2 UTF-8 CRLF HTML

- ❖ Attribute directives change the appearance or behavior of an element.



- ❖ ngClass directive adds and removes **CSS classes** on an HTML element.

- ❖ ngStyle is an attribute directive that updates **styles** for the HTML element.

EXPLORER

▼ FIRST-PROJECT

- JS test.js U
- TS test.ts U
- TS app-routing.mod... A
- # app.component.css M
- <> app.component.h... M
- TS app.component.s... A
- TS app.component.ts M
- TS app.module.ts M
- JS logging.service.js U
- TS logging.service.ts U
- > assets
- > environments
- ★ favicon.ico A
- <> index.html M
- TS main.ts M
- TS polyfills.ts A
- # styles.css A
- TS test.ts A
- ≡ .browserslistrc A

> OUTLINE

> TIMELINE

app.component.html M X TS app.component.ts M # app.component.css M

src > app > <> app.component.html > div

Go to component

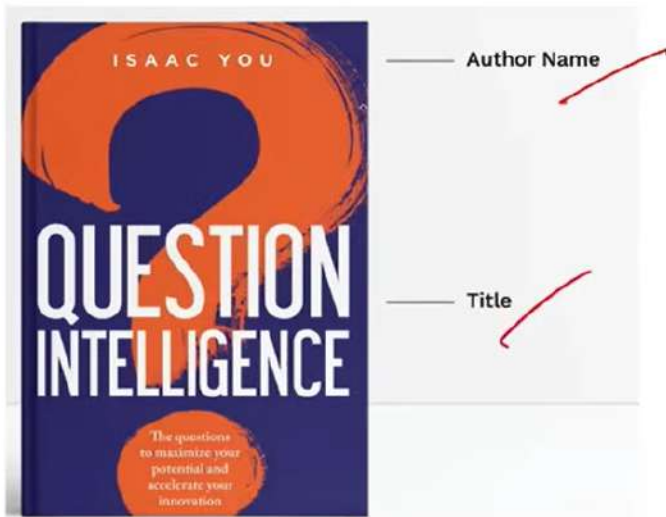
```

1  <!-- ngStyle is an attribute directive that updates styles for the HTML element. -->
2
3  <div [ngStyle]="{'background-color': colorName, 'color': 'white'}">I am ngStyle Happy</div>
4
5
6  <!-- ngClass directive adds and removes CSS classes on an HTML element. -->
7  <br>
8  <div [ngClass]="className">I am ngClass Happy</div>

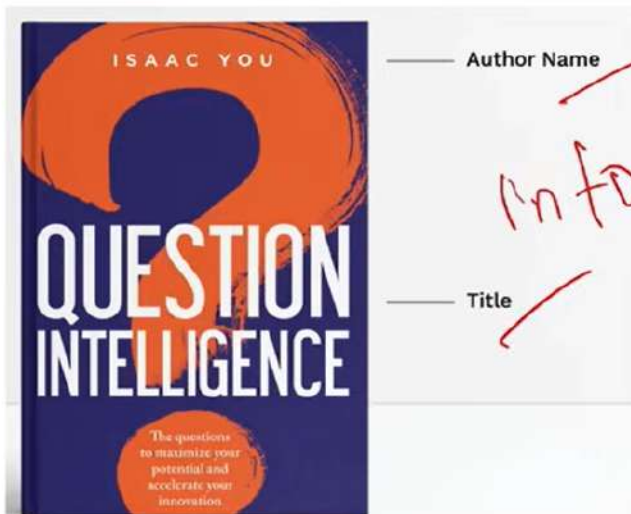
```

Component Directive	Structural Directive	Attribute Directive
1. Component directive is responsible for showing the first whole view. It is the most used one.	Structural directive is responsible for adding and deleting html elements in the view.	Attribute directive is responsible for changing the appearance of view by adding or removing styles/cssclasses of the html elements.
2. Starts with @ sign. Like: @Component	Starts with * sign. Like: *ngIf, *ngFor, *ngSwitch	Set inside square brackets. Like: [ngClass], [ngStyle]

- ❖ Angular decorators store **metadata** about a class, method, or property.
- ❖ Metadata is "data that provides information about other data".



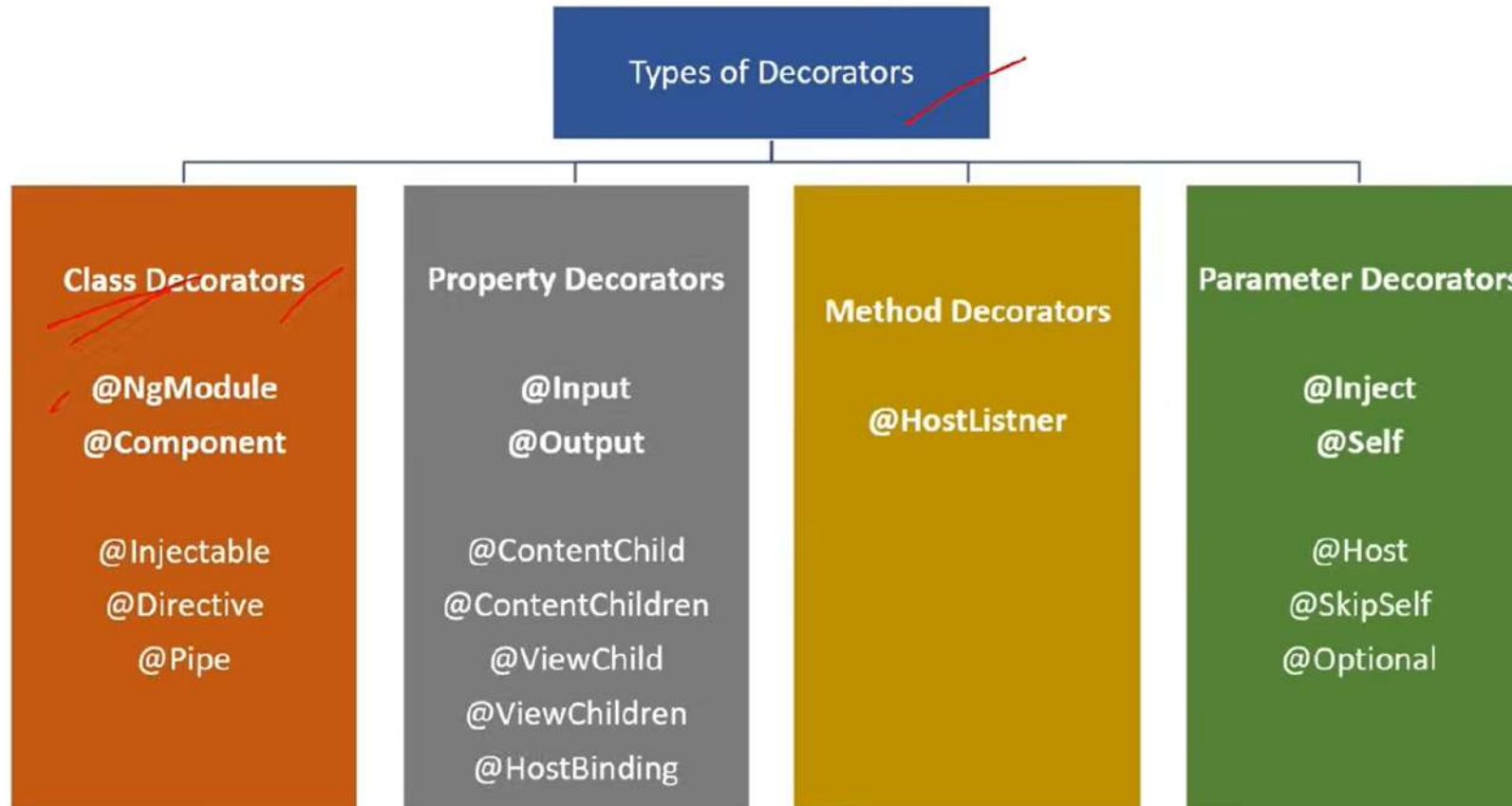
- ❖ Angular decorators store **metadata** about a class, method, or property.
- ❖ Metadata is "data that provides information about other data".



```
@Component({  
  selector: 'app-root',  
  templateUrl: './app.component.html',  
  styleUrls: ['./app.component.css']  
})  
  
export class AppComponent {  
  title = 'first-interview';  
}
```

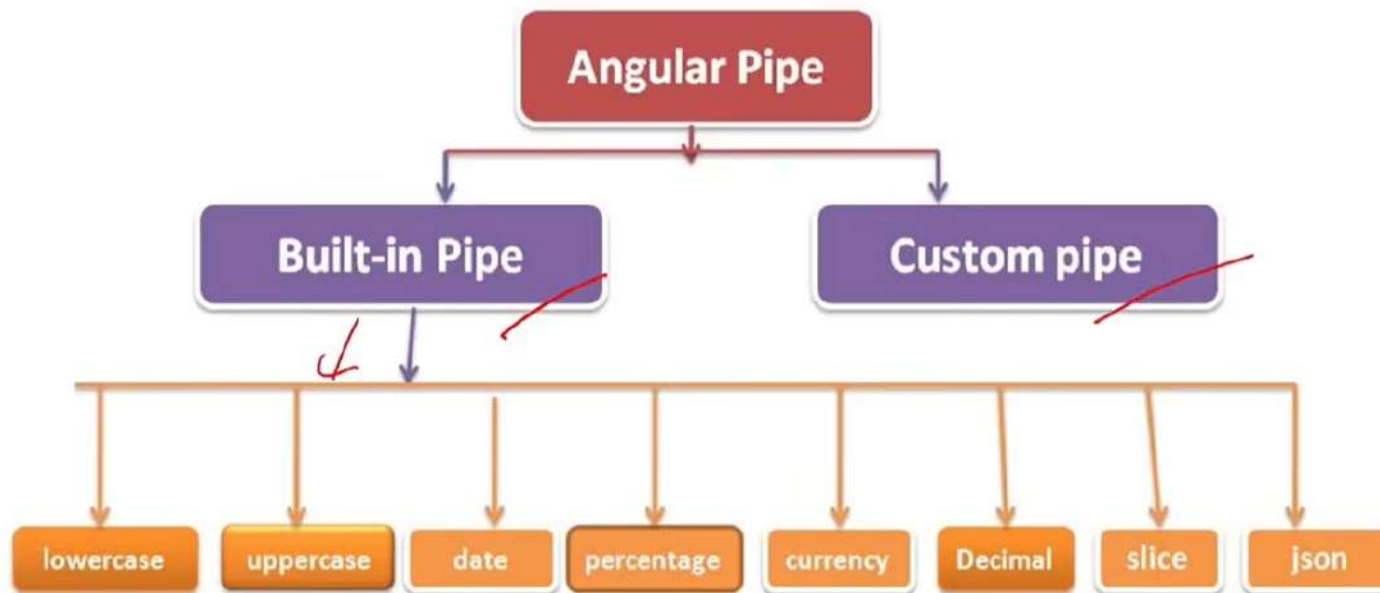
```
@NgModule({  
  declarations: [  
    AppComponent  
  ],  
  imports: [  
    BrowserModule,  
    AppRoutingModule  
  ],  
  providers: [],  
  bootstrap: [AppComponent]  
})  
  
export class AppModule { }
```

Metadata



What are Pipes? What are the types of Pipes & Parameterized Pipes?

- ❖ Pipes are simple functions which accept an input value and return a **transformed value**.



File Edit Selection View Go ... app.component.html - first-project - ...

EXPLORER

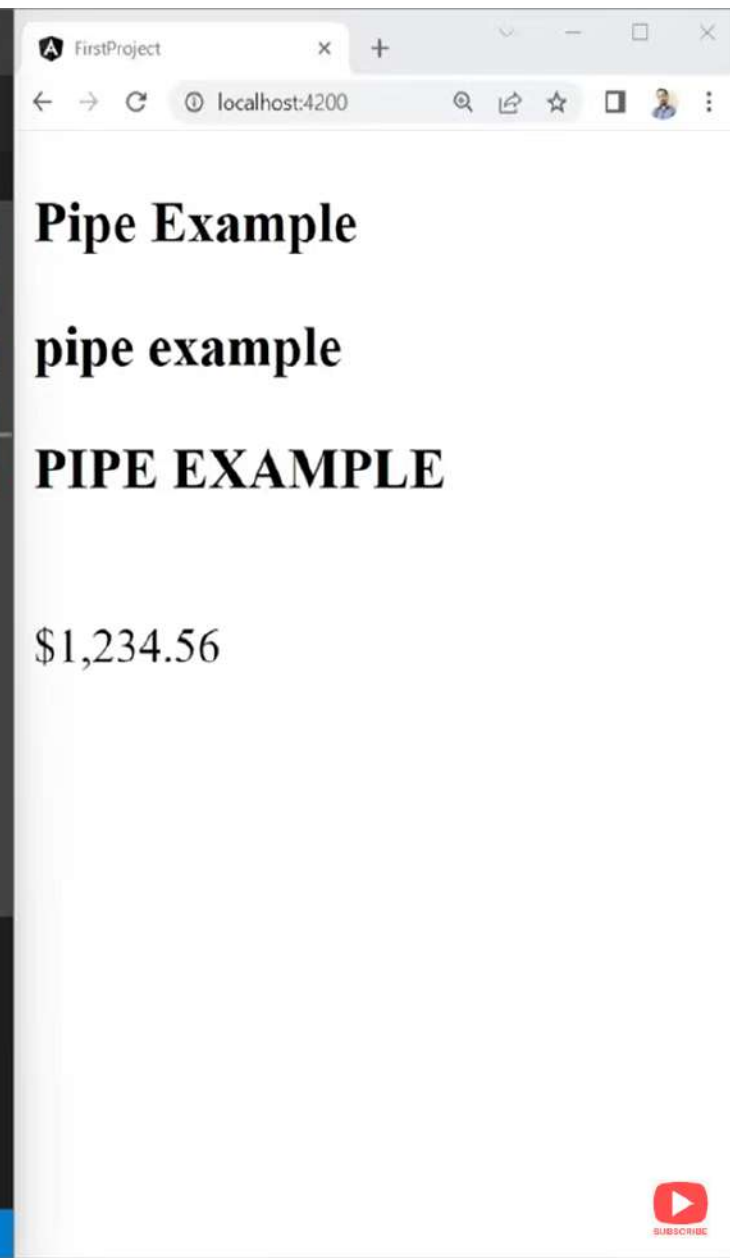
FIRST-PROJECT

- # app.component.css M
- <> app.component.html M
- TS app.component.spec.ts A
- TS app.component.ts M
- TS app.module.ts M
- > assets
- > environments
- ★ favicon.ico A
- <> index.html M
- TS main.ts M
- TS polyfills.ts A
- # styles.css A
- TS test.ts A
- ≡ .browserslistrc A
- ⚙ .editorconfig A
- 📄 .gitignore A
- { } angular.json M
- 📄 karma.conf.js A
- { } package-lock.json A
- > OUTLINE
- > TIMELINE

src > app > <> app.component.html > ...

```
1 <h3>{{title}}</h3>
2
3 <h3>{{title | lowercase}}</h3>
4
5 <h3>{{title | uppercase}}</h3>
6
7 <br>
8 {{ 1234.56 | currency }}
9
10
```

Ln 8, Col 11 (7 selected) Spaces: 4 UTF-8 CRLF HTML



6. Services & Dependency Injection

Q27. Explain Services with Example? **V. IMP.**

Q28. How to create Service in Angular?

Q29. How to use Dependency Injector with Services in Angular?

Q30. What is Hierarchical Dependency Injection? **V. IMP.**

Q31. What is Provider in Angular?

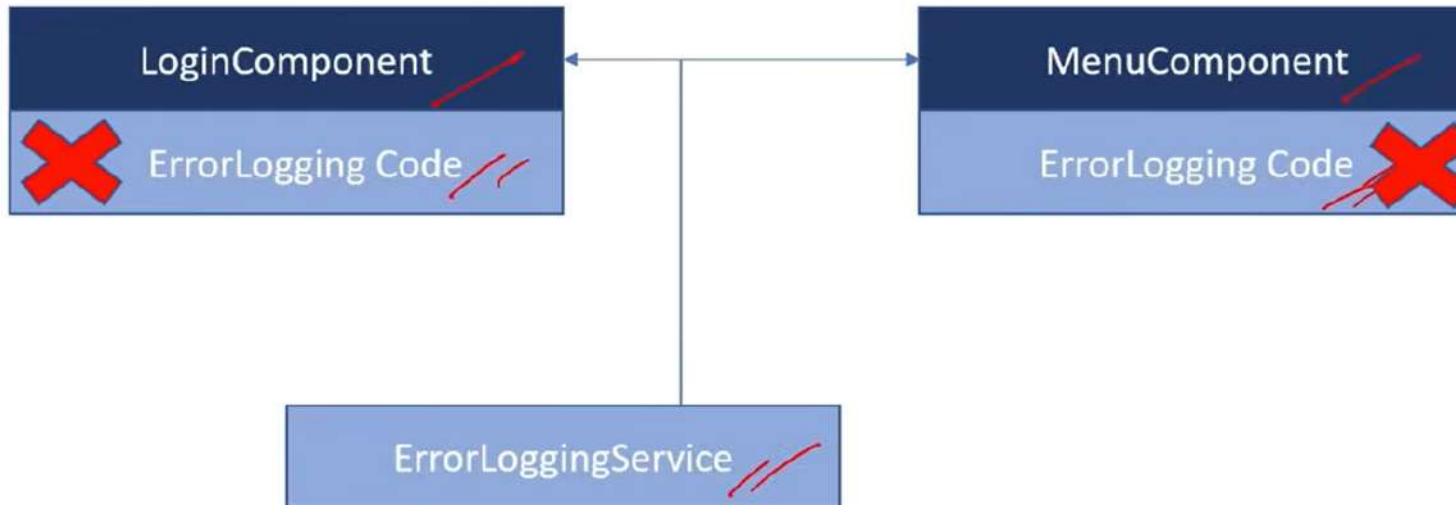
Q32. What is the role of @Injectable Decorator in a Service?



Q

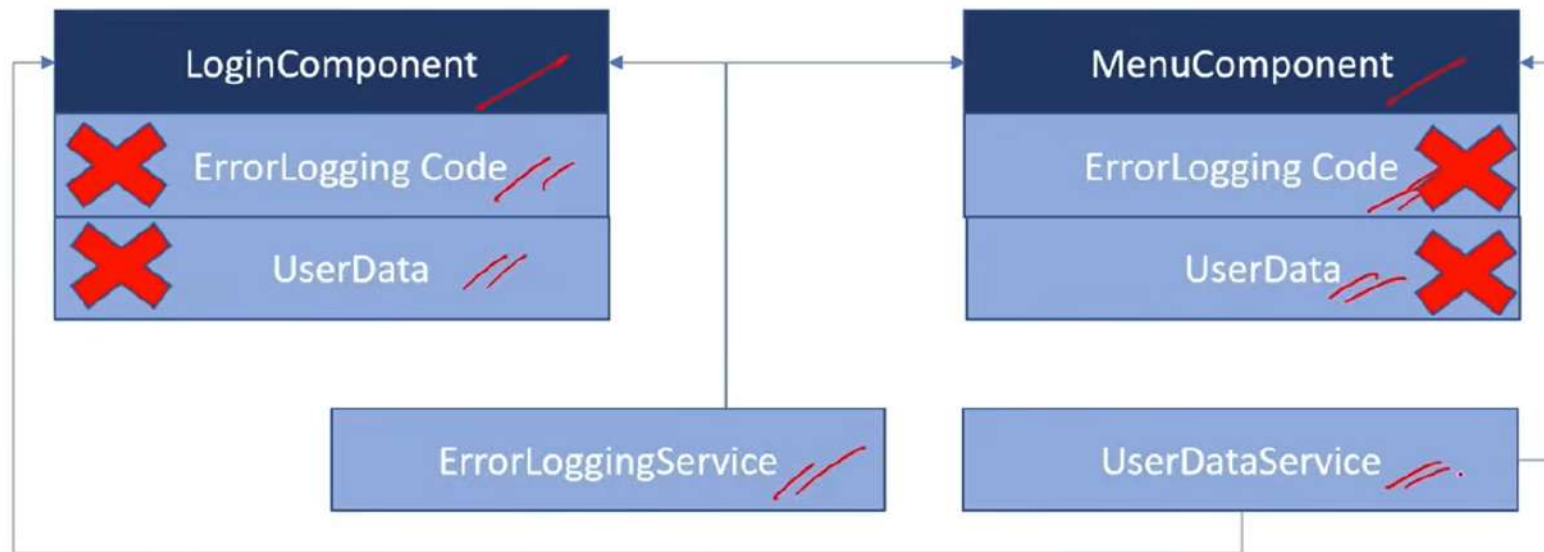
Explain Services with Example? **V. IMP.**

A



- ❖ A service is a **typescript class** and a **reusable code** which can be used in multiple components.





100%

- ❖ A service is a **typescript class** and a **reusable code** which can be used in multiple components.
- ❖ Service can be implemented with the help of **dependency injection**.

File Edit Selection View Go Run Terminal Help

login.component.ts - services-project - Visual Studio Code

EXPLORER

SERVICES-PROJECT

TS app-routing.module.ts A

app.component.css A

<> app.component.html M

TS app.component.spec.ts A

TS app.component.ts A

TS app.module.ts M

> assets

> environments

login

- # login.component.css U
- <> login.component.html U
- TS login.component.spec.ts U
- TS login.component.ts U

menu

- # menu.component.css U
- <> menu.component.html U
- TS menu.component.spec.ts U
- TS menu.component.ts U

Services

- TS logging.service.spec.ts U
- TS logging.service.ts U

★ favicon.ico A

<> index.html M

TS main.ts A

OUTLINE

TIMELINE

TS logging.service.ts U

TS login.component.ts U X

TS app.module.ts M

src > login > TS login.component.ts > LoginComponent

```
1 import { Component, OnInit } from '@angular/core';
2 import { LoggingService } from 'src/Services/logging.service';
3
4 @Component({
5   selector: 'app-login',
6   templateUrl: './login.component.html',
7   styleUrls: ['./login.component.css']
8 })
9 export class LoginComponent implements OnInit {
10
11   constructor(private loggingService: LoggingService) {
12
13     //console.log("Error Logged");
14
15     //const ls = new LoggingService();
16     //ls.LogError();
17
18     this.loggingService.LogError();
19   }
20
21   ngOnInit(): void {
22
23   }
24 }
25
26
```

Ln 7, Col 39

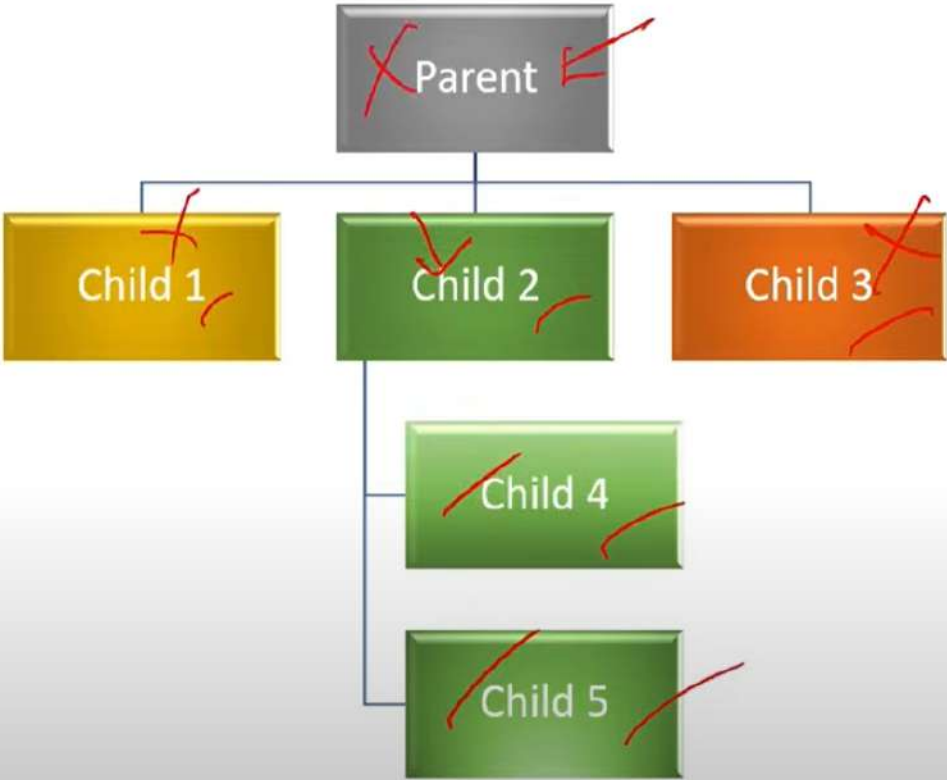
Spaces: 2

UTF-8

LF

{ } TypeScript

SUBSCRIBE



SUBSCRIBE

Visual Studio Code interface showing a TypeScript project named "services-project". The Explorer panel on the left displays the file structure, including components, services, and assets. The main editor displays the code for `app.module.ts`, which defines the `AppModule` class. The code includes imports for `LoggingService`, `AppRoutingModule`, and `AppComponent`. The `@NgModule` decorator is configured with declarations, imports, providers, and bootstrap options. The status bar at the bottom indicates the current position is Line 14, Column 18 (51 selected), with 2 spaces, UTF-8 encoding, and LF line endings.

File Edit Selection View Go Run Terminal Help

app.module.ts - services-project - Visual Studio Code

EXPLORER

SERVICES-PROJECT

- TS app-routing.module.ts A
- # app.component.css A
- <> app.component.html M
- TS app.component.spec.ts A
- TS app.component.ts A
- TS app.module.ts M
- > assets
- > environments
- login
 - # login.component.css U
 - <> login.component.html U
 - TS login.component.spec.ts U
 - TS login.component.ts U
- menu
 - # menu.component.css U
 - <> menu.component.html U
 - TS menu.component.spec.ts U
 - TS menu.component.ts U
- Services
 - TS logging.service.spec.ts U
 - TS logging.service.ts U
- ★ favicon.ico A
- <> index.html M
- TS main.ts A

OUTLINE

TIMELINE

src > app > TS app.module.ts > AppModule

```
import { LoggingService } from src/services/logging.service ;
6
7 import { AppRoutingModule } from './app-routing.module';
8 import { AppComponent } from './app.component';
9
10 @NgModule({
11   declarations: [
12     AppComponent,
13     LoginComponent,
14     MenuComponent
15   ],
16   imports: [
17     BrowserModule,
18     AppRoutingModule
19   ],
20   providers: [LoggingService],
21   bootstrap: [AppComponent, LoginComponent, MenuComponent]
22 })
23 export class AppModule { }
24
```

C:\services-project\src\Services\logging.service.ts • Untracked

Ln 14, Col 18 (51 selected) Spaces: 2 UTF-8 LF {} TypeScript



What is Provider in Angular?

- ❖ A provider is an object declared inside decorators which inform Angular that a particular service is available for injecting inside the components.

```
10 @NgModule({
11   declarations: [
12     AppComponent,
13     LoginComponent,
14     MenuComponent
15   ],
16   imports: [
17     BrowserModule,
18     AppRoutingModule
19   ],
20   providers: [LoggingService],
21   bootstrap: [AppComponent, LoginComponent]
22 })
23 export class AppModule { }
24
```

```
@Injectable({
  providedIn: 'root'
})
export class LoggingService {

  constructor() { }

  LogError()
  {
    console.log("Error Logged1");
    console.log("Error Logged2");
    console.log("Error Logged3");
  }
}
```

```
@Component({
  selector: 'app-login',
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.css'],
  providers: [LoggingService]
})
export class LoginComponent implements OnInit {

  constructor(private loggingService: LoggingService) { }

  ngOnInit() {
    this.loggingService.LogError();
    //const ls = new LoggingService();
    //ls.LogError();
    // console.log("Error Logged1");
    // console.log("Error Logged2");
    // console.log("Error Logged3");
  }
}
```


7. Lifecycle-Hooks

Q33. What are Parent-Child Components?

Q34. What are Lifecycle Hooks in Angular?

Q35. What is a Constructor in Angular?

Q36. What is ngOnInit life cycle hook in Angular?

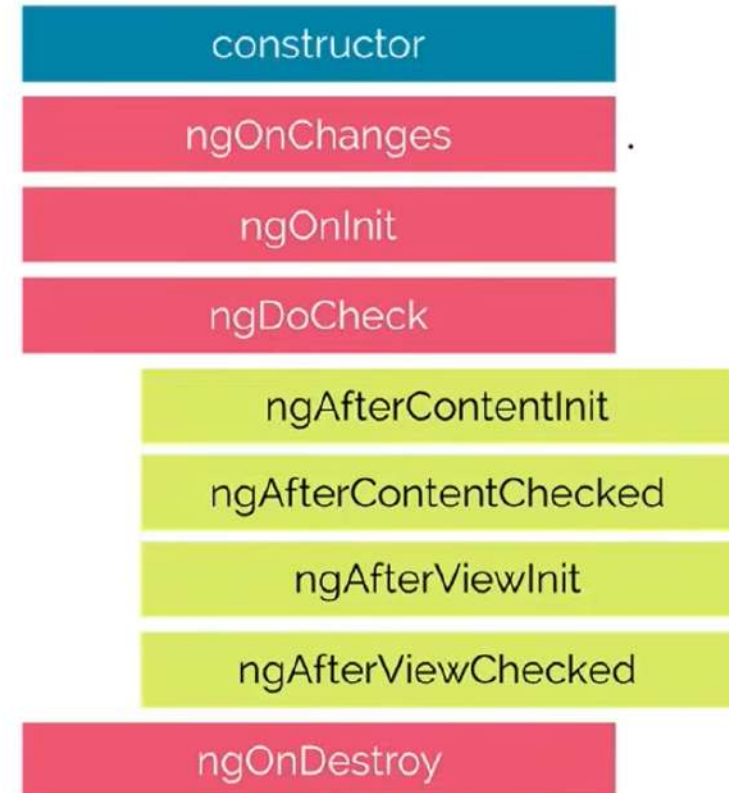
Q37. What is the difference between constructor and ngOnInit? **V. IMP.**



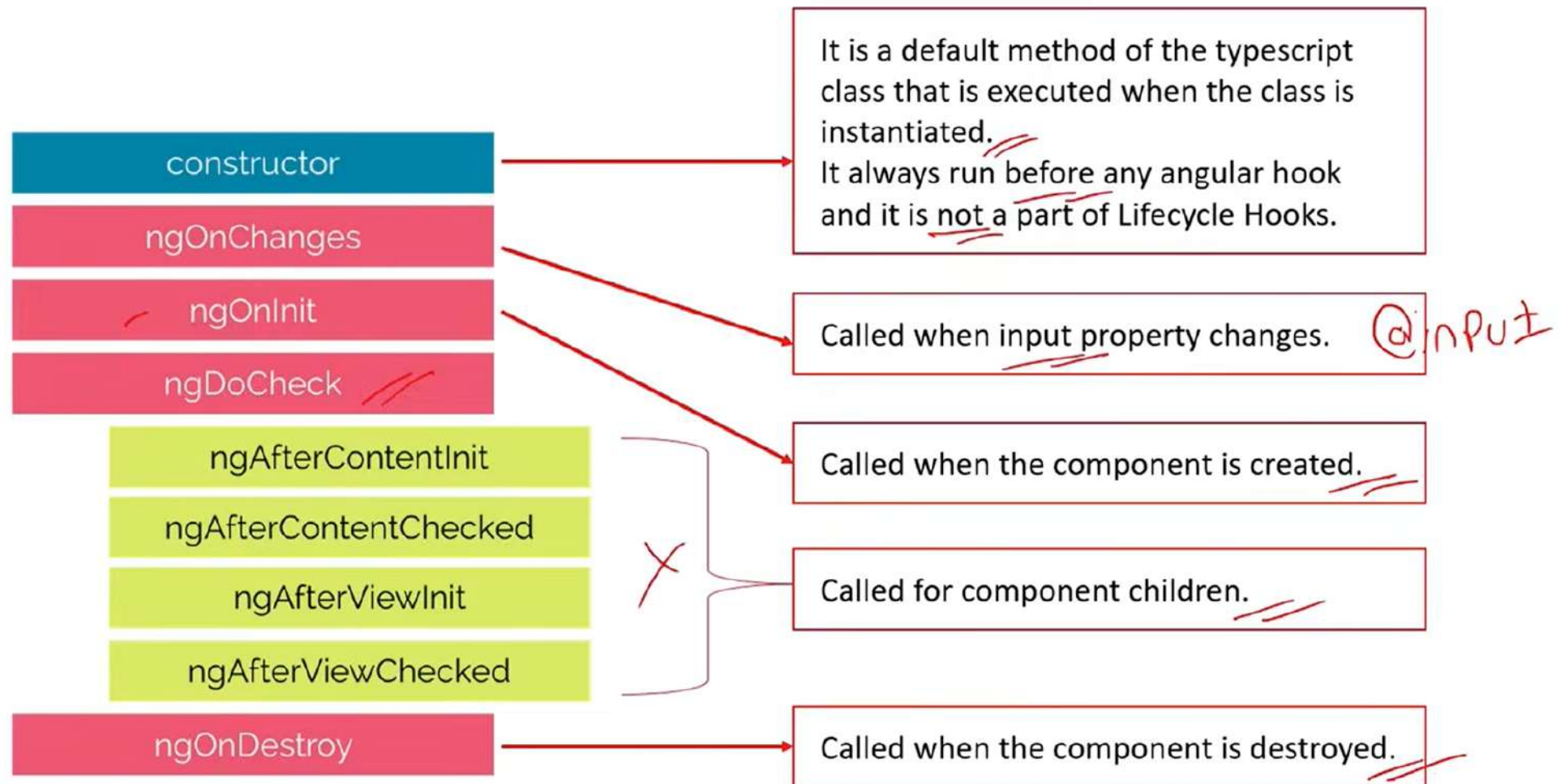
- ❖ A component from creation to destruction goes through several stages and these stages are the life cycle hooks.
- ❖ The stages will cover activities like:
 - Component instantiating.
 - Rendering the component html view.
 - Creating the child components(if required).
 - Destroying the component.

What are Lifecycle Hooks in Angular? **V. IMP.**

- ❖ A component from creation to destruction goes through several stages and these stages are the life cycle hooks.
- ❖ The stages will cover activities like:
 - Component instantiating.
 - Rendering the component html view.
 - Creating the child components(if required).
 - Destroying the component.



What are Lifecycle Hooks in Angular? **V. IMP.**



- ❖ The constructor is a method in a **TypeScript** class that automatically gets called when the class is being **instantiated**.
- ❖ Constructor always runs before any angular hook and it is not a part of Lifecycle Hooks.
- ❖ Constructor is widely used to inject dependencies(services) into the component class.

```
TS sample.component.ts U X
src > app > sample > TS sample.component.ts > ...
1  import { Component, OnInit } from '@angular/core';
2
3  @Component({
4    selector: 'app-sample',
5    templateUrl: './sample.component.html',
6    styleUrls: ['./sample.component.css']
7  })
8  export class SampleComponent implements OnInit {
9
10     constructor() {}
11
12     ngOnInit(): void {
13     }
14
15  }
16
```

What is ngOnInit life cycle hook in Angular?

1. ngOnInit signals the activation of the created component.
2. This is the second hook and called after ngOnChanges.
3. ngOnInit called only once during lifecycle.

```
@Component({  
  selector: 'app-sample',  
  templateUrl: './sample.component.html',  
  styleUrls: ['./sample.component.css']  
})  
export class SampleComponent implements OnInit {  
  
  constructor() { }  
  
  ngOnInit(): void {  
  }  
}
```



ngOnInit	Constructor
1. ngOnInit is an Angular lifecycle hook , which signals the activation of the created component.	The constructor is a method in a TypeScript class , that automatically gets called when the class is being instantiated.
2. ngOnInit is called after ngOnChanges lifecycle-hook.	Constructor is called before any lifecycle-hook.
3. When ngOnInit called, everything about component is already ready, so it's use is to perform most of the business logic on component.	When constructor called, everything in component is not ready, so it's mostly used for injecting dependencies only.

8. Observable\ HttpClient\ RxJS

Q38. What are Asynchronous operations?

Q39. What is the difference between Promise and Observable? **V. IMP.**

Q40. What is RxJS?

Q41. What is Observable? How to implement Observable **V. IMP.**

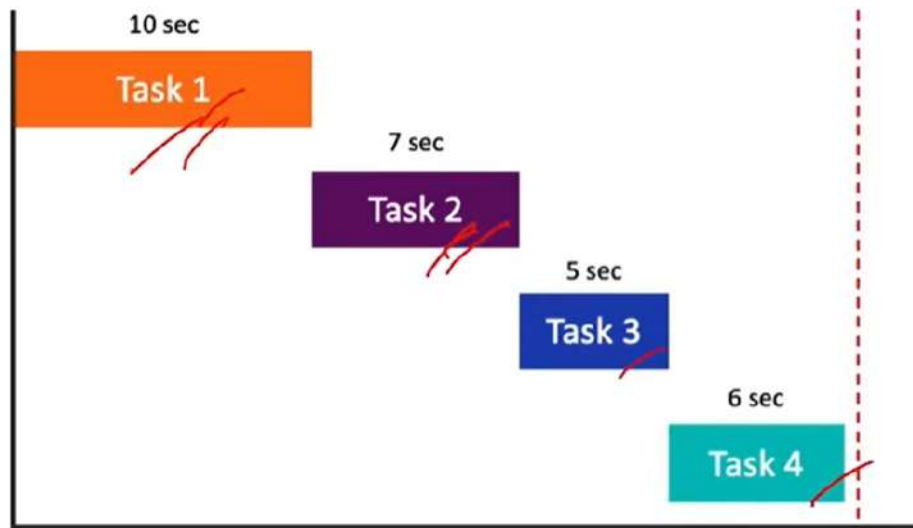
Q42. What is the role of HttpClient in Angular?

What are Asynchronous operations?

- ❖ Observables are used to perform asynchronous operations and handle asynchronous data

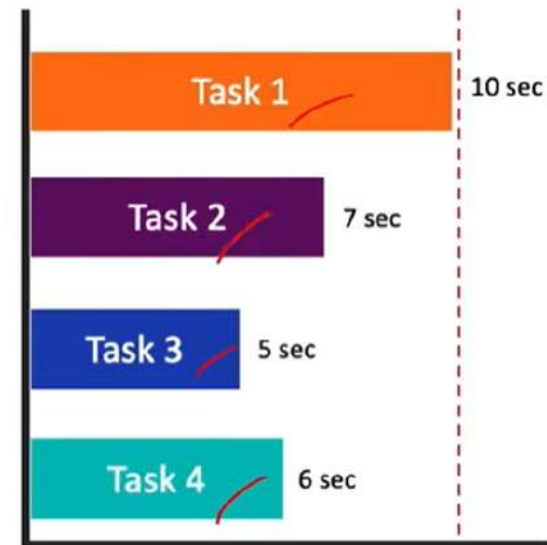
SYNCHRONOUS

JS

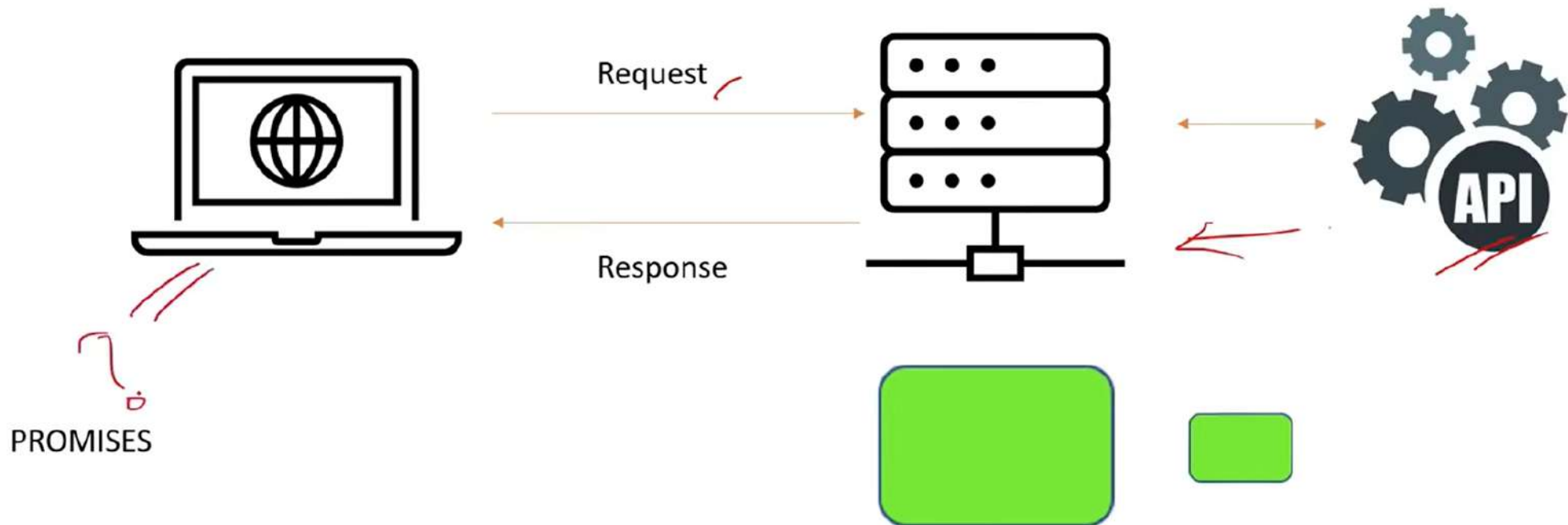


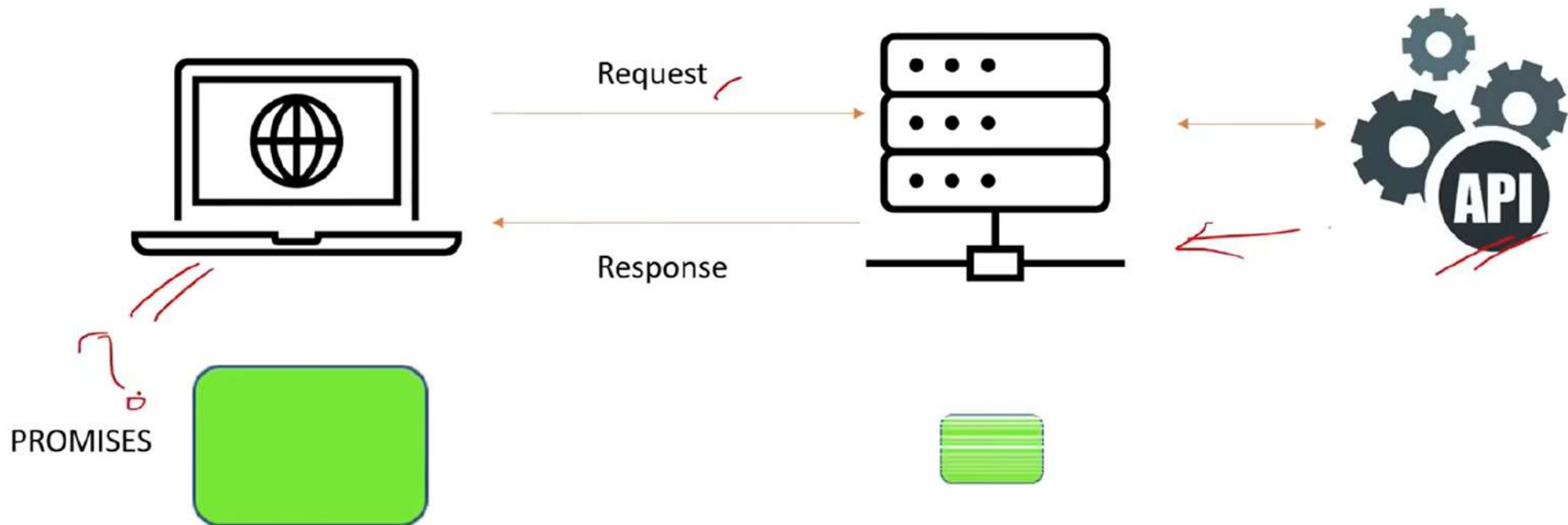
Time taken (28 sec)

ASYNCHRONOUS



Time taken (10 sec)

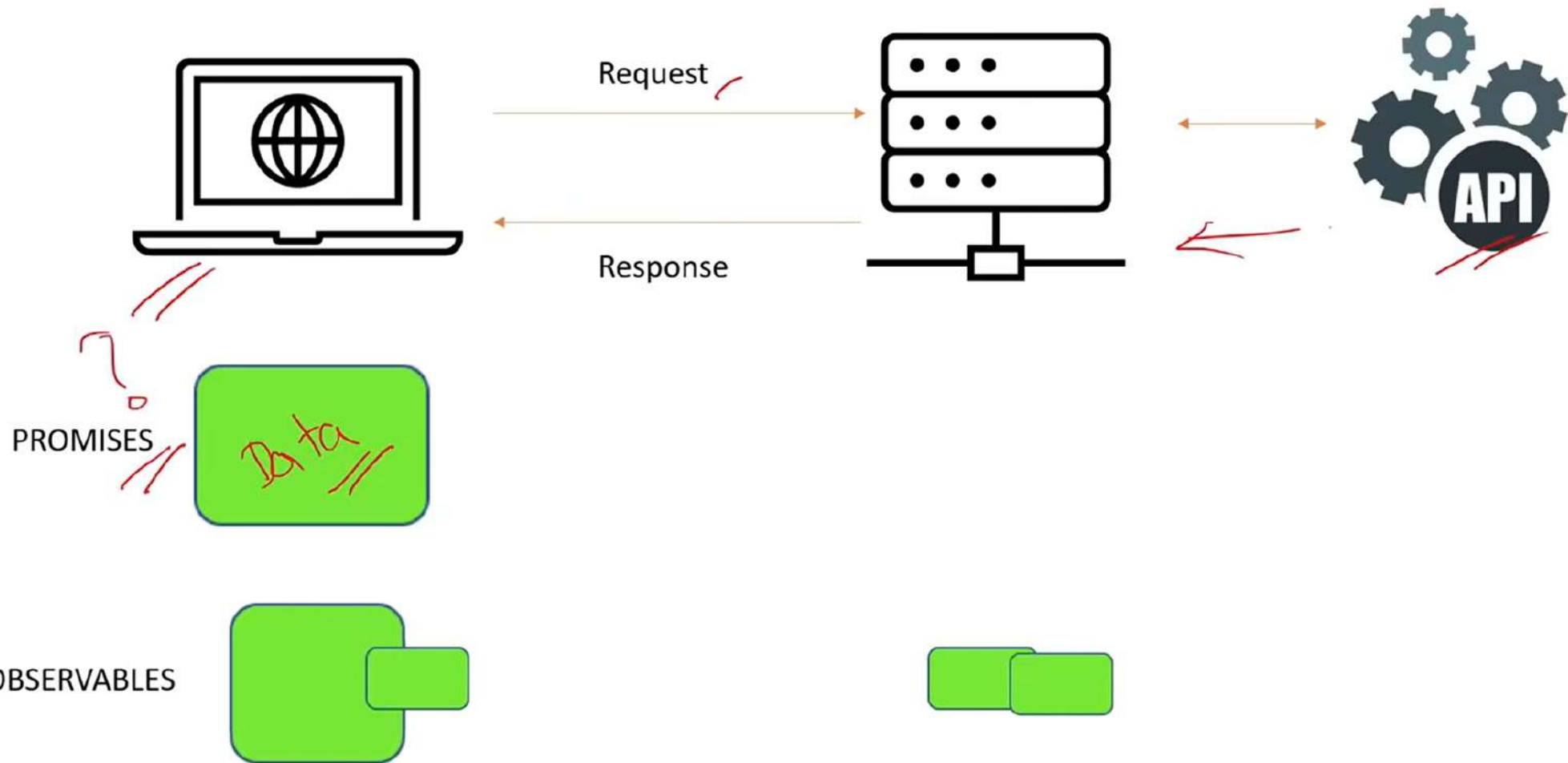




Q

What is the difference between Promise and Observable? **V. IMP.**

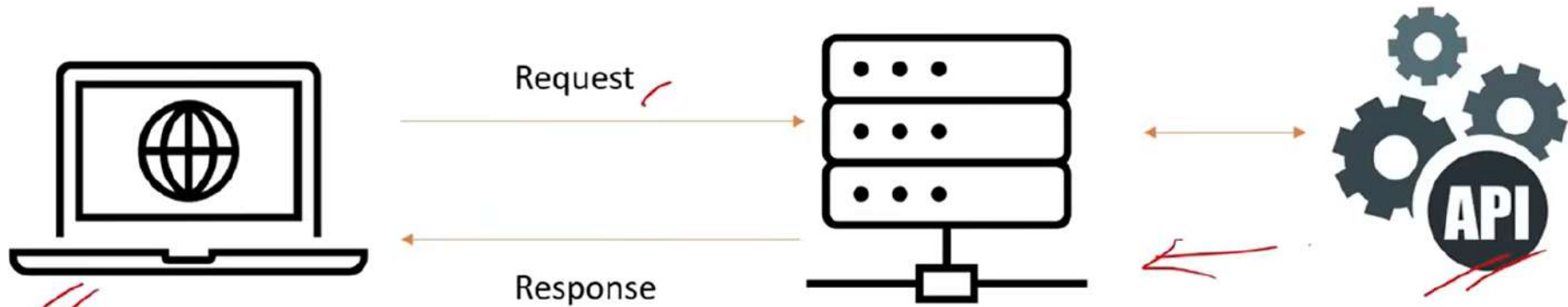
A



Q

What is the difference between Promise and Observable? **V. IMP.**

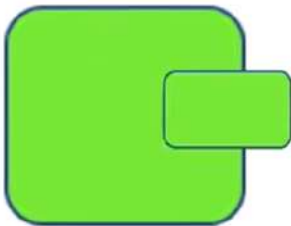
A



PROMISES



OBSERVABLES



What is the difference between Promise and Observable?

Observables	Promises
1. Emit multiple values over a period of time. Also called streaming of data.	Emit a single value at a time.
2. Are lazy: they're not executed until we subscribe to them using the <code>subscribe()</code> method. <i>memory</i>	Are not lazy: execute immediately after creation.
3. Have subscriptions that are cancellable using the <code>unsubscribe()</code> method.	Are not cancellable . <i>Code</i>



The screenshot shows a web browser with a single tab titled "RxJS - Introduction". The address bar shows the URL "rxjs.dev/guide/overview". The page has a dark red header with a hamburger menu icon, the RxJS logo, and navigation links for "OVERVIEW", "REFERENCE", and "TEAM". The main content area has the heading "Introduction" and a paragraph of text. A red box highlights the first sentence of the paragraph, and a red arrow points to the word "sequences" within it.

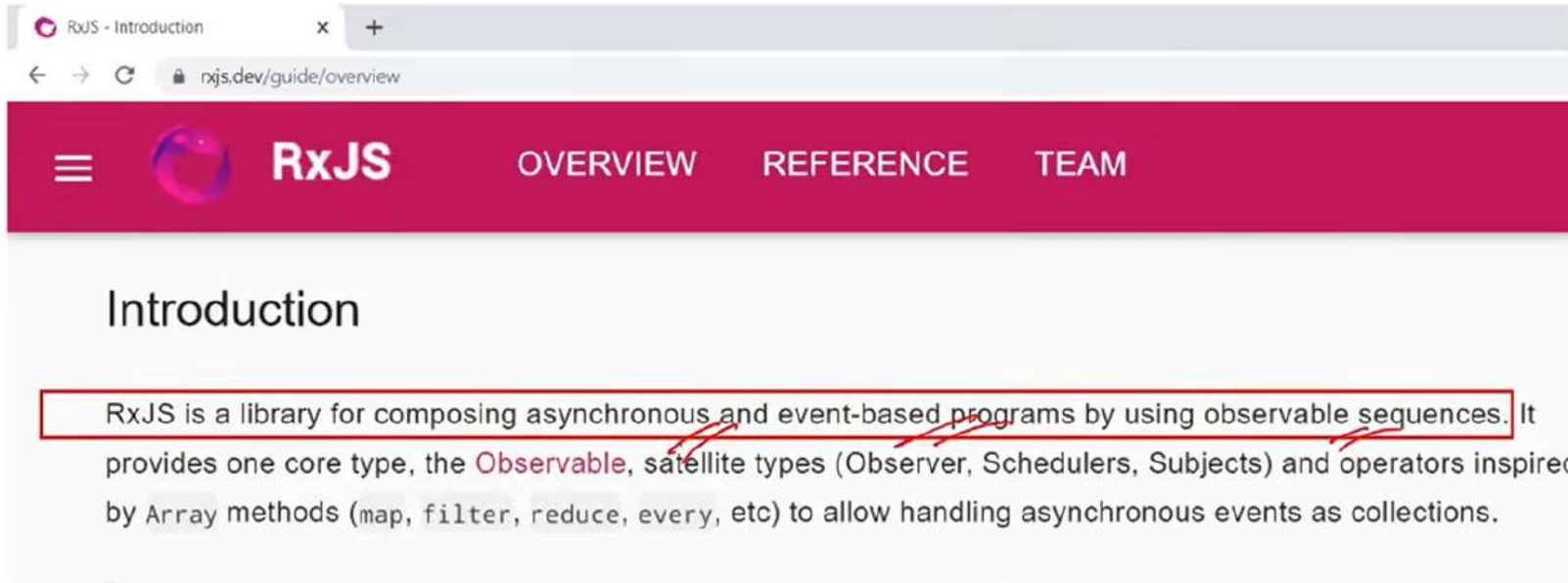
RxJS - Introduction x +

← → ↻ rxjs.dev/guide/overview

☰ RxJS OVERVIEW REFERENCE TEAM

Introduction

RxJS is a library for composing asynchronous and event-based programs by using observable sequences. It provides one core type, the **Observable**, satellite types (Observer, Schedulers, Subjects) and operators inspired by Array methods (map, filter, reduce, every, etc) to allow handling asynchronous events as collections.

A screenshot of a web browser showing the RxJS Introduction page. The browser's address bar displays 'rxjs.dev/guide/overview'. The page has a dark red header with the RxJS logo and navigation links for 'OVERVIEW', 'REFERENCE', and 'TEAM'. The main content area is titled 'Introduction' and contains a paragraph describing RxJS as a library for composing asynchronous and event-based programs using observable sequences. The first sentence of this paragraph is highlighted with a red box and red underlines.

RxJS - Introduction x +

← → ↺ rxjs.dev/guide/overview

☰ RxJS OVERVIEW REFERENCE TEAM

Introduction

RxJS is a library for composing asynchronous and event-based programs by using observable sequences. It provides one core type, the **Observable**, satellite types (Observer, Schedulers, Subjects) and operators inspired by Array methods (map, filter, reduce, every, etc) to allow handling asynchronous events as collections.

- ❖ RxJS is a javascript library, that allow us to work with asynchronous data stream with the help of observables.
- ❖ Observables introduced by **RxJS** library.

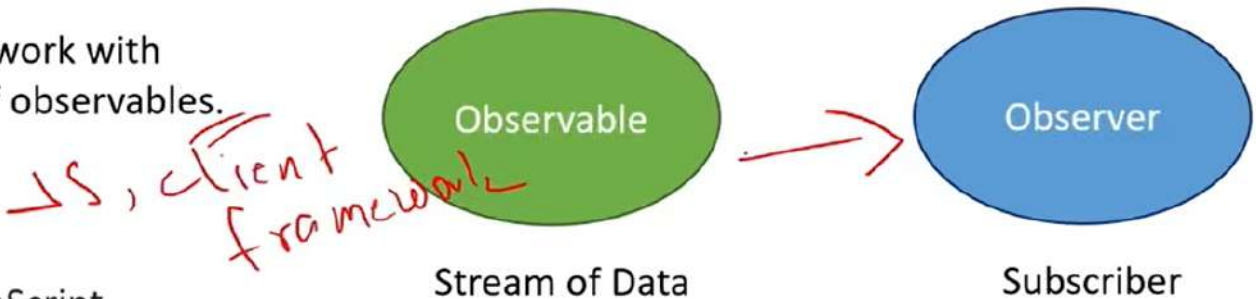
What is RxJS?



❖ RxJS is a javascript library, that allow us to work with asynchronous data stream with the help of observables.

❖ Observables introduced by **RxJS** library.

❖ RxJS stands for Reactive Extensions for JavaScript.



- ❖ Observables are used to **stream data** to multiple components. *///*
- ❖ It's a 3 step process:

1. Import Observable from RxJS Library.

2. Create Observable & Emit Data.

3. Finally subscribe the Data.

TS app.component.ts M X

src > app > TS app.component.ts > AppComponent > ngOnInit

```
1 import { Component, OnInit } from '@angular/core';
2 import { Observable } from 'rxjs';
3
4 @Component({
5   selector: 'app-root',
6   templateUrl: './app.component.html',
7   styleUrls: ['./app.component.css']
8 })
9 export class AppComponent implements OnInit {
10
11   title = 'observable-project';
12
13
14   myObservable = new Observable(observer =>{
15     console.log("observable start");
16     observer.next("a");
17     observer.next("b");
18     observer.next("c");
19   });
20
21   ngOnInit(): void {
22
23     this.myObservable.subscribe((val => {
24       console.log(val);
25     }));
26
27   }
```

1. Import Observable from RxJS Library.

2. Then Create Observable & Emit Data.

3. And finally subscribe the Data.



SUBSCRIBE

TS app.component.ts M X

src > app > TS app.component.ts > AppComponent > isPromis

```
1  import { Component, OnInit } from '@angular/core';
2  import { Observable } from 'rxjs';
3
4  @Component({
5    selector: 'app-root',
6    templateUrl: './app.component.html',
7    styleUrls: ['./app.component.css']
8  })
9  export class AppComponent implements OnInit {
10
11    title = 'observable-project';
12
13    isPromis = new Promise<string>((resolve, reject) => {
14      console.log("promise start");
15    });
16
17    myObsesrvable = new Observable(observer =>{
18      console.log("observable start");
19      observer.next("a");
20      observer.next("b");
21      observer.next("c");
22    });
23
24    ngOnInit(): void {
25
26
27
28
```

1. Import Observable from RxJS Library.
2. Then Create Observable & Emit Data.
3. And finally subscribe the Data.

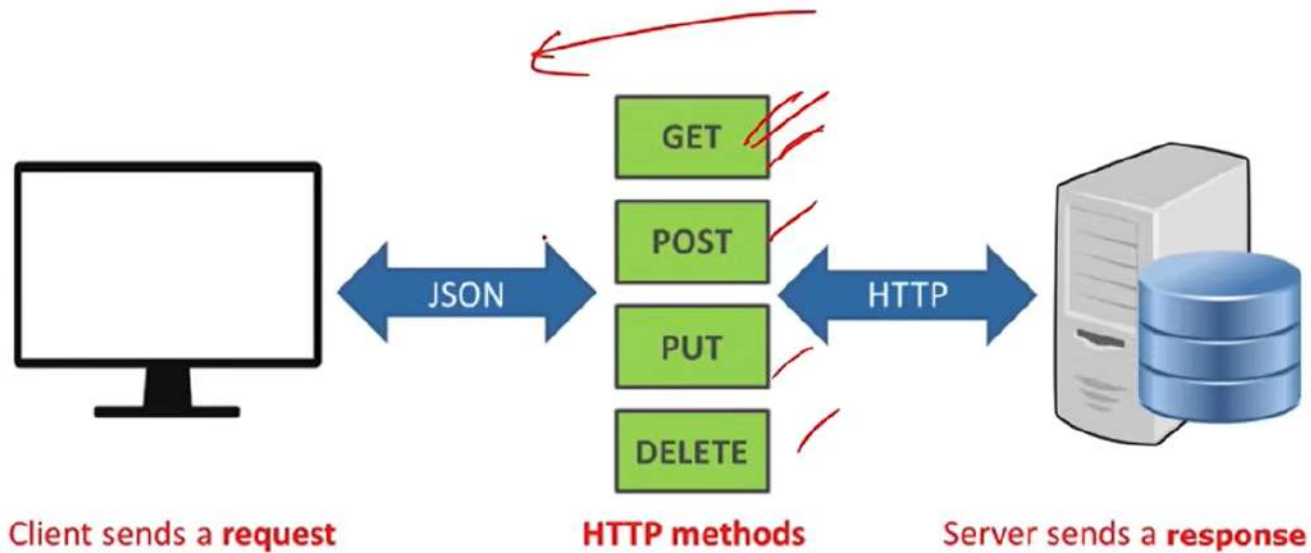


SUBSCRIBE

What is the role of HttpClient in Angular?

- ❖ HttpClient is a **built-in service** class available in Angular.
- ❖ @angular/common/http package.
- ❖ Performs HTTP requests.

*Services
↓
Component*



9. Typescript-Basics

Q43. What is Typescript? Or What is the difference between Typescript and Javascript?

Q44. What is the difference between let and var keyword?

Q45. What is Type annotation?

Q46. What are Built in/ Primitive and User-Defined/ Non-primitive Types in Typescript?

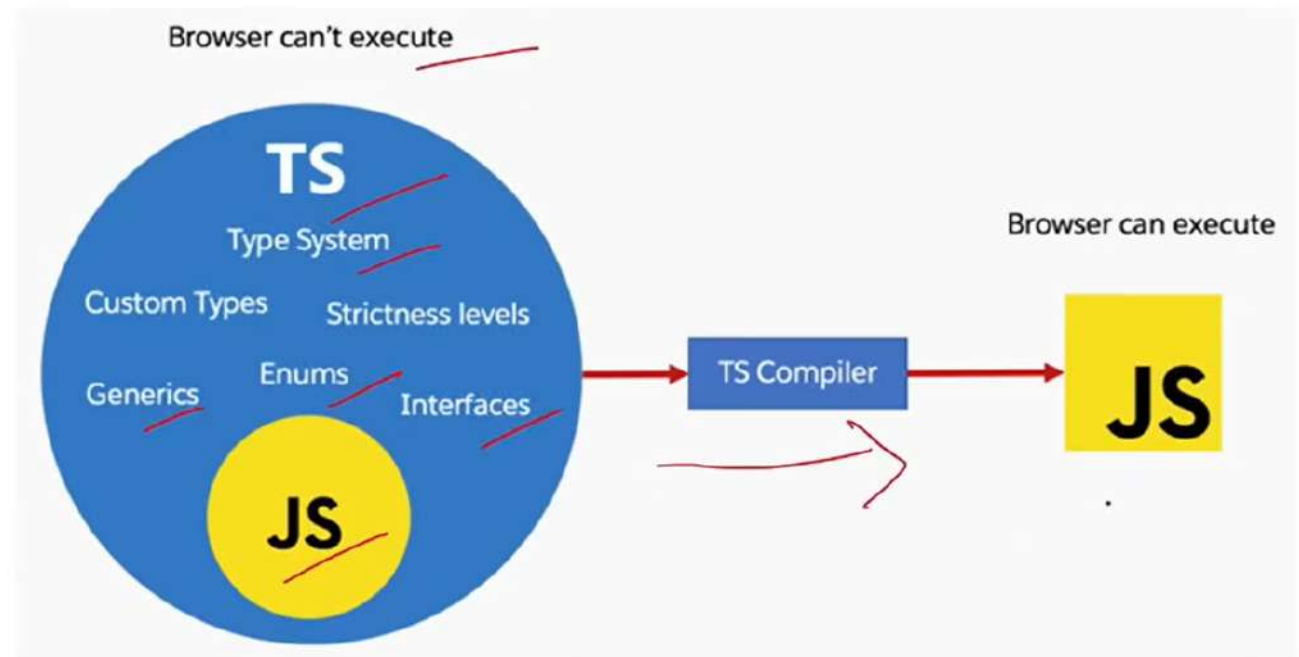
Q47. What is “any” type in Typescript? **V. IMP.**

Q48. What is Enum type in Typescript?

Q49. What is Type Assertion in Typescript?

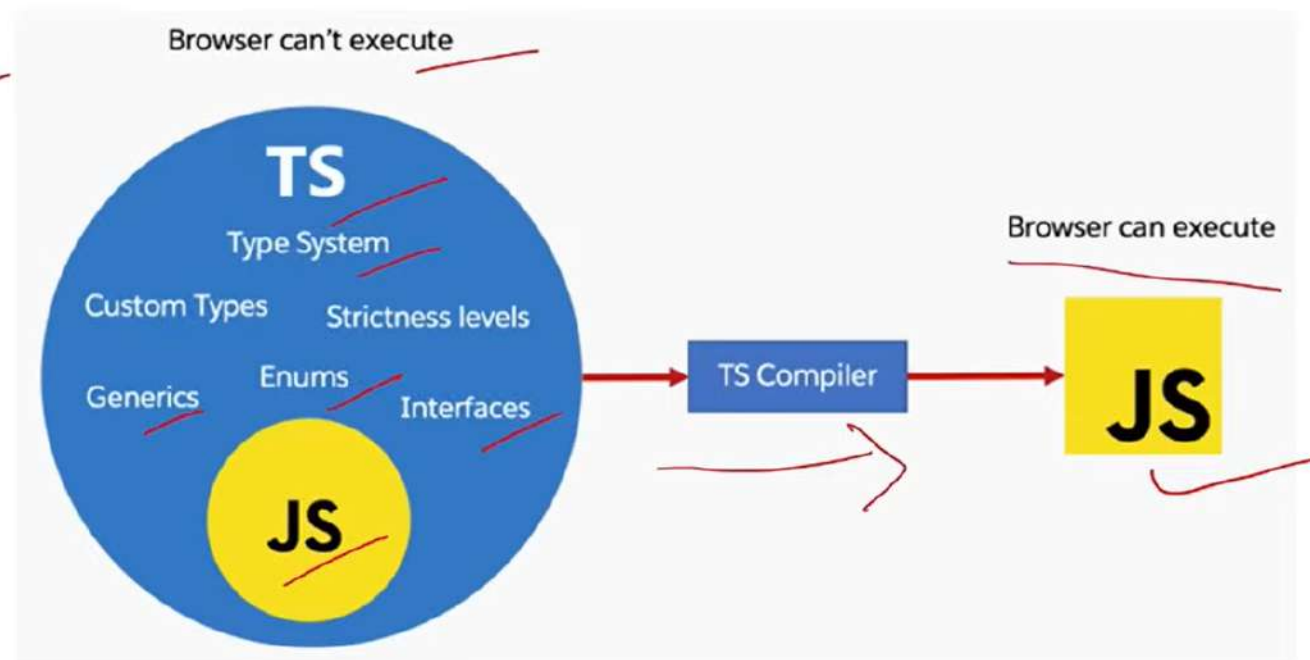
Q50. What are Arrow Functions in Typescript?

What is Typescript? What are the advantages of Typescript over Javascript?



What is Typescript? What are the advantages of Typescript over Javascript?

1. Typescript is a strongly typed language
2. Typescript is a superset of JavaScript
3. It has Object oriented features
4. Detect error at compile time



EXPLORER

TYPESCRIPT-PROJECT

Classes-Objects.ts

Encapsulation.js

Encapsulation.ts

Enum.js

Enum.ts

Implement-Extend.ts

Inheritance.ts

Interface.ts

let-var.js

let-var.ts

main.js

main.ts

Multiple-Inheritance.ts

Polymorphism.js

Polymorphism.ts

stronglytype.js

test.ts

TypeAnnotations.ts

TypeAssertion.ts

types.ts

void-never.ts

OUTLINE

TIMELINE

TS stronglytype.ts 1 X

TS stronglytype.ts > ...

```
1 // 1. Typescript is a strongly typed language
2
3 // 2. Typescript is a superset of JavaScript
4
5 // 3. It has Object oriented features
6
7 // 4. Detect error at compile time
8
9
10 // strongly typed
11 let someValue: string = "interview happy";
12
13 someValue = 5;
14
15
```

PROBLEMS 1

OUTPUT

DEBUG CONSOLE

TERMINAL

Found 1 error.

PS C:\Typescript-Project> tsc stronglytype.ts stronglytype.js

error TS6504: File 'stronglytype.js' is a JavaScript file. Did you mean to enable the 'allowJs' option?

The file is in the program because:

Root file specified for compilation

Found 1 error.

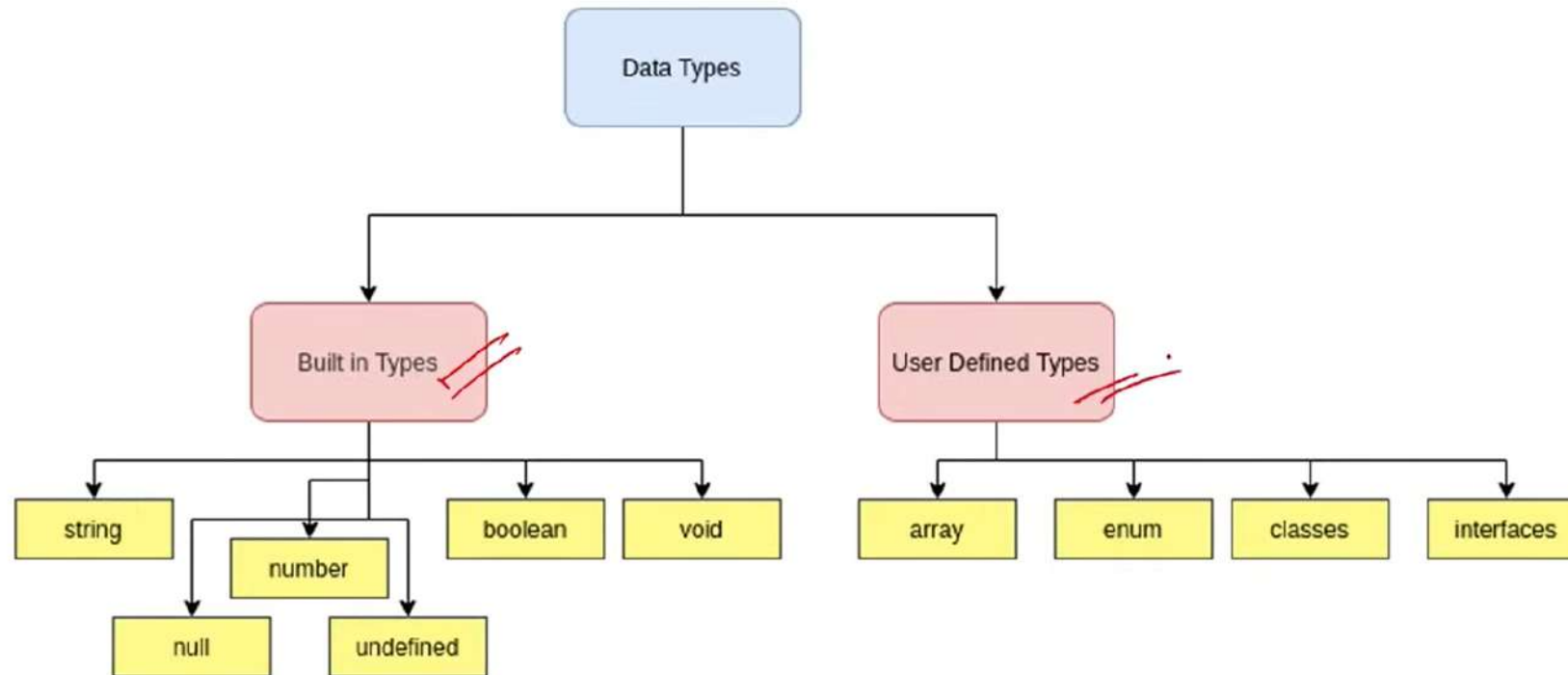
PS C:\Typescript-Project>

powershell + v

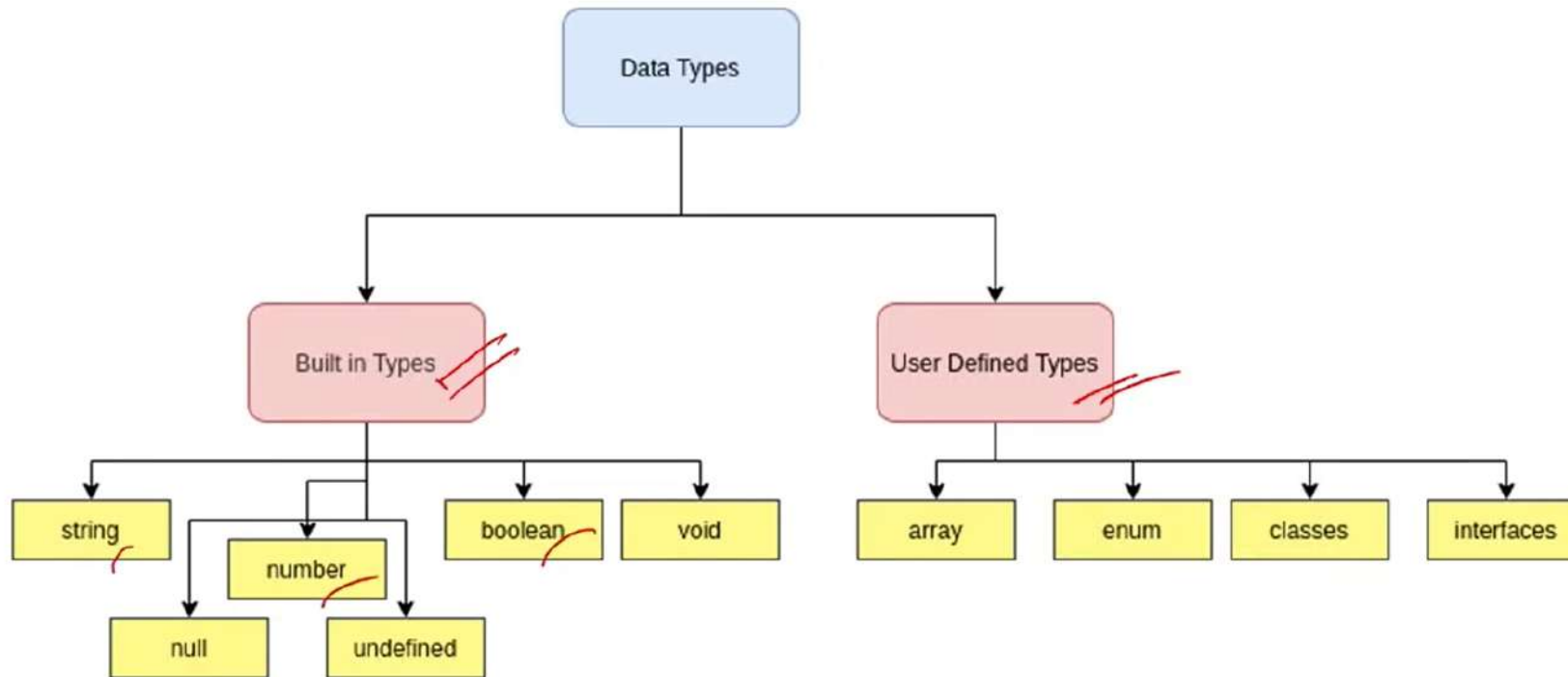
1 0

Ln 13, Col 15 (16 selected) Spaces: 4 UTF-8 CRLF {} TypeScript

What are Built in/ Primitive and User-Defined/ Non-primitive Types in Typescript?



What are Built in/ Primitive and User-Defined/ Non-primitive Types in Typescript?



- ❖ Built-in Data types are used to store simple data.
- ❖ User-Defined types are used to store complex data.



Get Started

TS types.ts X



C: > Typescript-Project > TS types.ts > [🔗] list

```
1  //Basic/ Primitive Types in Typescript
2
3  //Boolean
4  let isCompleted: boolean = false;
5
6  //Number
7  let count: number = 100;
8
9  //String
10 let myname: string = "Happy";
11
12 //Array
13 let list: number[] = [1, 2, 3];
14
15
16
17
18
```

Search: Typescript-Project > Typescript
1. types.ts
2. types.ts
3. types.ts
4. types.ts
5. types.ts

0 0

Ln 13, Col 30 (7 selected)

Spaces: 4

UTF-8

CRLF

{ } TypeScript





Get Started

TS types.ts 2 X



C: > Typescript-Project > TS types.ts > ...

```
10 let myname: string = "Happy";
11
12 //Array
13 let list: number[] = [1, 2, 3];
14
15
16 //Tuple
17 // Declare a tuple type
18 let x: [string, number];
19
20 // Initialize it
21 x = ["Happy", 10]; // OK
22
23 // Initialize it incorrectly
24 x = [10, "Happy"]; // Error
25
26
27
```

```
Typescript-Project > TS types.ts > ...
let myname: string = "Happy";
//Array
let list: number[] = [1, 2, 3];
//Tuple
// Declare a tuple type
let x: [string, number];
// Initialize it
x = ["Happy", 10]; // OK
// Initialize it incorrectly
x = [10, "Happy"]; // Error
```



⊗ 2 ⚠ 0

Ln 25, Col 1 Spaces: 4 UTF-8 CRLF {} TypeScript





Get Started

TS Enum.ts X



C: > Typescript-Project > TS Enum.ts > ...

```
1  
2 //Defintion - Enums allows to define a set of named constants.  
3
```

```
4 const DirectionNorth = 0;
```

```
5 const DirectionSouth = 1;
```

```
6 const DirectionWest = 2;
```

```
7 const DirectionEast = 3;
```

```
8  
9 let dir = DirectionEast;
```

```
10  
11 enum Direction {
```

```
12     North = 1,
```

```
13     South,
```

```
14     West,
```

```
15     East
```

```
16 }
```

```
17  
18 let dirNew = Direction.East;
```



0 0

Ln 2, Col 36 Spaces: 2 UTF-8 CRLF {} TypeScript





Get Started

TS TypeAssertion.ts X



C: > Typescript-Project > TS TypeAssertion.ts > ...

```
1
2 //Definition: Type assertion is a technique
3 //that informs the compiler about the type of a variable.
4
5 let myname;
6
7 myname = "Happy";
8
9 let isEndCharY = (<string>myname).endsWith("y");
10
11
```

0 0 0

Ln 3, Col 58

Spaces: 4

UTF-8

CRLF

{ } TypeScript





Get Started

TS ArrowFunctions.ts X



C: > Typescript-Project > TS ArrowFunctions.ts > [y]

```
let x = function(a, b) {  
  a * b;  
}
```



```
1  
2 //Arrow functions are the shorthand syntax  
3 //for defining the anonymous function  
4  
5 let x = function(a, b)  
6 {  
7   a * b;  
8 }  
9  
10 let y = (a, b) => a * b;  
11  
12  
13  
14
```



0 0 0

Ln 10, Col 18 Spaces: 4 UTF-8 CRLF {} TypeScript

